IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF MATHEMATICS

MSCI RESEARCH PROJECT

A Variational Approach to American Option Pricing via Physics-Informed Neural Networks

Author:
James Wu

Supervisor(s):
Harry Zheng

Submitted in partial fulfillment of the requirements for the MSci in Mathematics at Imperial College London

June 7, 2026

Abstract

American option pricing requires solving free boundary PDEs with no closed-form solution. We reformulate the problem as a variational inequality and propose a physics-informed neural network (PINN) approach to solve it. On both the Black-Scholes and Heston models, our PINNs match tree-based benchmarks to within 3% relative error. We extend the approach to multi-asset settings where we benchmark against equivalent one-dimensional reductions. We characterise the free boundary from the learned solution, yielding interpretable early-exercise surfaces. The results establish PINNs as a practical alternative to lattice methods for higher-dimensional problems in financial mathematics.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Professor Harry Zheng, for his expert guidance and continued support throughout this project. Our weekly meetings and his insights were invaluable in the development of this research.

I am also grateful to Yun Zhao for his continued assistance, especially in the numerical implementation side of the project. His help was crucial in overcoming the technical challenges we faced.

Finally, I would like to thank my family for their patience and encouragement throughout what has been a challenging degree. They have provided a constant source of motivation, which I deeply appreciate.

Plagiarism statement and use of AI

The work contained in this thesis is my own work unless otherwise stated.

The use of AI tools in this dissertation is limited. I used ChatGPT and Claude to help me understand some of the mathematical concepts and methods used, but my research was done manually through published, peer-reviewed sources, all of which are cited in the bibliography. My implementation of the methods in Python are initially done without AI, but are later debugged and optimised with the help of Claude Code.

Signature: James Wu
Date: June 7, 2026

Contents

List of Figures

Chapter 1

Introduction

1.1 Motivation

In addition to bonds and stocks, the financial market offers derivatives, which are financial instruments whose value depends on the value of an underlying asset, which could be a stock, a bond, an index or even another derivative. These are useful in the real world for hedging and speculation, widely used by institutions and individuals alike, and are mathematically rich objects that have been the subject of intense research for decades.

Options are a specific type of derivative contract that gives the holder the right, but not the obligation, to buy or sell an underlying asset at a predetermined strike price K, specified at the time of option purchase. These contracts have a fixed expiry time T, at which the holder must decide whether to exercise the option or let it expire worthless. Different types of options exist: European options can only be exercised at the expiry time T, while American options can be exercised at any time before expiration. With the increased flexibility of American options, they are more interesting mathematical objects to study, since they are much more difficult to price. Tackling this problem is exactly the focus of this project. Holders of American options must constantly ask themselves whether they should settle for the current value by exercising the option immediately, or wait for a potentially better opportunity.

We will concentrate our attention on put options (which give the holder the right to sell the underlying asset) in this project. This is because the American option is equivalent to the European option for call options (which give the holder the right to buy the underlying asset) under the assumption of no dividends, and we do not consider dividends in this project. At expiry, the payoff of the put option is given by $\max(K - S_T, 0)$, where S_T is the price of the underlying asset at the time of expiry T. This is because holders of the put option should exercise the option when the price of the underlying asset is below the strike price. Similarly, for a call option, holders should buy when the price $S_T > K$, and hence the payoff is given by $\max(S_T - K, 0)$. If the underlying asset price is such that we should exercise the option, the option is in-the-money (expires with a positive payoff). Otherwise, the option is out-of-the-money (expires worthless). Figure 1.1 illustrates these payoff functions at expiry with the grey line.

Figure 1.1 also shows the value of a put and a call option at the initial time, which can be decomposed into the intrinsic value (the payoff should it be exercised immediately) and the time value (the additional value from the possibility of future price movements). We can see clearly that the European option price bounds the American option price from below, since the holder of the European option cannot exercise early. Interestingly, we can see that the value of the European put is lower than the intrinsic value for in-the-money asset prices. This happens

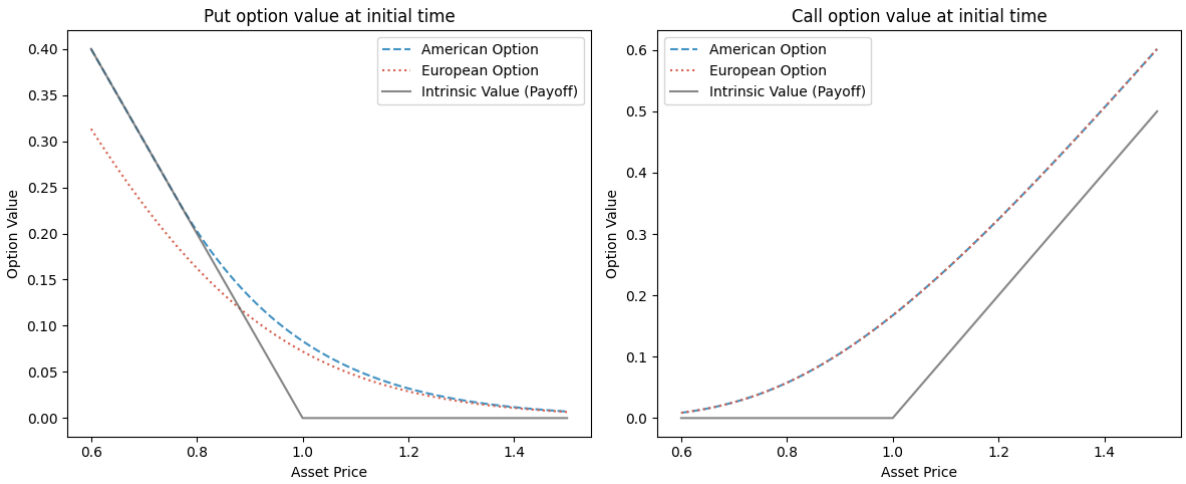

Figure 1.1 Option values (for put and call) at initial time

when we have a positive interest rate, and the holder has to wait until expiry to receive the payoff, so the price is discounted. Of course, the intrinsic value bounds the American option from below, since the holder can always choose to exercise immediately.

These options are very important financial instruments, and tens of millions of contracts are traded every day for the S&P 500 index alone. For that reason, it is important to be able to price these options accurately. This is a non-trivial task, as the price of an option depends on many factors, such as the underlying asset price. This is generally modelled as a stochastic process, and the American option does not admit a closed-form solution.

1.2 Literature review

The pioneering work in option pricing was done by Black and Scholes [1], who derived a closed-form solution for the price of the European call option under the assumption that the underlying asset price follows a geometric Brownian motion. This model, known as the Black-Scholes model, is considered the foundational work in option pricing theory, and the authors were awarded the Nobel Prize in Economics in 1997 for their contribution. However, it relies on several assumptions, such as a constant risk-free rate and a constant volatility, which are not always realistic in practice. Real markets exhibit different implied volatilities (the volatility parameter that, when input into the Black-Scholes formula, yields the market price of the option) for different strike prices and maturities. This phenomenon is known as the volatility smile (see Figure 1.2 for an illustration of the smile in the real financial market), indicating the insufficiency of the Black-Scholes model in capturing market dynamics.

As a result, people started developing more sophisticated models that took inspiration from the Black-Scholes model. Heston [2] modelled volatility as a stochastic process in itself, which is more consistent with empirical observations of financial markets, and derived the European option prices for this proposed model. This model is more flexible and provides a better fit to the empirical volatility smile.

While the European option has an analytical solution for specific, well-researched models, the American option is much more difficult to price, since it allows the holder to exercise the option at any time before expiration, making it analogous to an optimal stopping problem. These optimisation problems do not admit a closed-form solution in general. As a result, various numerical methods have been developed to tackle this problem, such as the binomial tree method

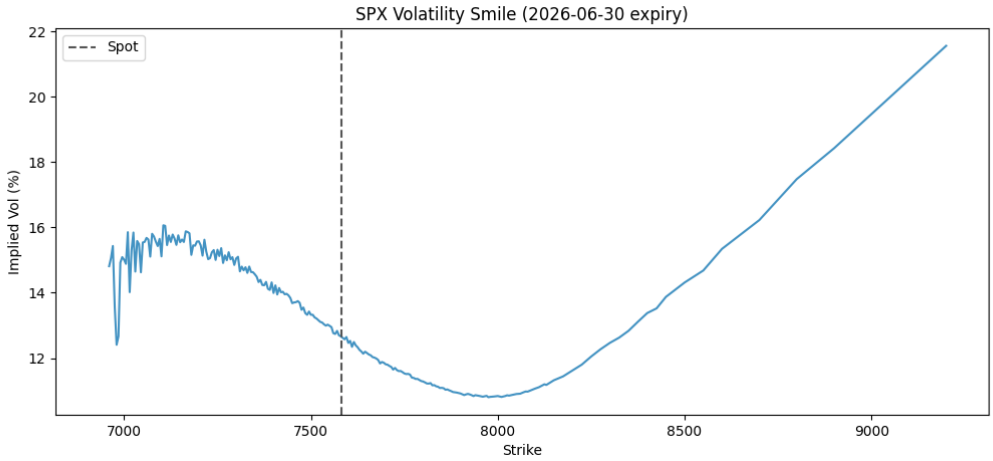

Figure 1.2 Volatility smile for SPX options with expiry on 30 June 2026. Data from Yahoo Finance (retrieved on 1 June 2026).

[3, 4] for the Black-Scholes and Heston models respectively. In these methods, the asset price is discretised into a tree structure and the option prices are computed via backwards induction from the terminal payoff at expiry. Monte-Carlo simulation methods were also explored [5], where heavy research has gone into optimising the idea of pricing an option by simulating an ensemble of paths and computing the average value. More recently, variational methods have been explored, where the optimal stopping problem is formulated as a variational inequality, with deep neural networks finding approximations to the solution [6, 7], borrowing from the fields of machine learning and physics. This method is further adapted to incorporate fractional Sobolev norms to guarantee convergence of the solution to the true option price instead of just pointwise convergence, which has been particularly developed for American option pricing [8].

1.3 Overview of project

This project aims to numerically solve the American option pricing problem under the Black-Scholes and Heston models on increasingly complex payoffs by iteratively building benchmarks.

For each individual model, we begin by deriving the price of a European call option, which serves as a benchmark for the binomial tree method. These formulae are well-known and can be found in the original papers [1, 2], but we provide a condensed derivation for completeness, with clearer justification of the results employed in the steps in the derivation.

The binomial trees can easily be adapted to price American options, which provide a benchmark for the neural network solution. The binomial tree method is a well-established numerical technique for pricing options, but we take an interpolation approach, as suggested in [4], for the Heston model where there are two concurrent processes (the asset and the volatility) to build a tree for. The original method yields good results, but we further adapt the method to allow efficient pricing across the whole pricing domain.

We implement the PINN framework for a neural network solution to the American option pricing problem, trained on a loss function we construct which is inspired by the boundary behaviour of the option price, and with weights tuned by adaptive weighting schemes. We additionally implement an alternative loss function based on the fractional Sobolev norm, an original research field explored in [8] by one of my supervisor's PhD students, which we adapt to price American options under the whole domain space instead of just at-the-money as in the original paper.

We use both the neural network and the binomial tree method as benchmarks for the multi-asset case, where we price put options on the product of assets, which has yet to be explored in the literature in depth. This is motivated by the fact that there exists an equivalent single-asset reduction for this payoff, which allows us to validate the results against the benchmarks already established in the one-asset case. With this foundation established, we are then free to tackle alternative payoffs where such a reduction does not exist, such as the maximum or minimum of several assets, and still have confidence in the results since the methods have been validated on the simpler cases.

We will conclude our numerical results by applying our solution to a characterisation of the free boundary. The free boundary is the critical asset price at which the holder of the option is indifferent between exercising immediately and waiting, which we compute numerically (e.g. with binary search) for deterministic methods such as the binomial tree. For non-deterministic methods, we propose a novel method to frame the free boundary problem as a soft-classification problem, where we make distributional assumptions on our continuation values and use the empirical variance of the continuation values as a measure of uncertainty that tunes the soft classification. The continuation values are estimated through Monte-Carlo simulation, and the soft classification is done through an empirically tuned function.

The project aims to provide a comprehensive numerical framework for pricing American options under two different models, with detailed numerical results highlighting the strengths and shortcomings of each method, and insights into the option price behaviour under different payoffs, complexity and market conditions. All of these are backed by a rigorous mathematical framework that borrows from the fields of stochastic calculus, numerical analysis and machine learning, allowing a translation of theoretical insights into practical numerical methods that can be applied in real-world scenarios.

All of the methods explored in this project are implemented in Python, with the code available on GitHub[1].

[1]This repository can be accessed with the URL `github.com/jameswu5/M4R`.

Chapter 2

Mathematical Background

2.1 Preliminaries

We will first assemble the necessary mathematical background to understand the option pricing problem and build some foundations into the methods we will be using to solve it. The definitions and results are mostly standard, and can be found in textbooks such as [9, 10] but we will cover them here for self-containment and to set up notation for the rest of the project. We will only cover the main concepts and not delve too deeply into the details, since our focus is more on the numerical methods and their applications.

2.1.1 Probabilistic Setup

The option pricing problem is inherently probabilistic, as we are uncertain about the future evolution of the underlying asset price. Therefore, we need a rigorous mathematical framework to model this uncertainty. The formalisation of the underlying randomness in our model for the market will be done through *probability spaces*, and the arrival of information in the market through time through *filtrations*.

Definition 2.1.1 (Probability space)**.** A *probability space* is a triple $(\Omega, \mathcal{F}, \mathbb{P})$ where Ω is the sample space, $\mathcal{F}$ is a σ-algebra on Ω, and $\mathbb{P} : \mathcal{F} \to [0, 1]$ is a probability measure such that $\mathbb{P}(\Omega) = 1$.

Definition 2.1.2 (Filtration, filtered probability space)**.** A *filtration* on the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ is a non-decreasing family $(\mathcal{F}_t)_{t\geq 0}$ of sub-σ-algebras of $\mathcal{F}$, that is to say, $\mathcal{F}_s \subseteq \mathcal{F}_t \subseteq \mathcal{F}$ for all $s \leq t$. The quadruple $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t\geq 0}, \mathbb{P})$ is called a *filtered probability space*.

We will also define a *martingale*, which formalises the idea that the future expectation of a process is equal to the current value (there is no drift in the process), which is a key concept in the risk-neutral pricing framework.

Definition 2.1.3 (Martingale)**.** A process $M = (M_t)_{t\geq 0}$ is a *martingale* with respect to the filtration $\{\mathcal{F}_t\}$ and the probability measure $\mathbb{P}$ if $\mathbb{E}[M_t \mid \mathcal{F}_s] = M_s$ for all $0 \leq s \leq t$.

The Markov assumption is also commonly made in financial modelling, which states that the future evolution of the process only depends on the current state and not on the past history. It is useful because it greatly simplifies models, and hence we will employ it in our project as well.

Definition 2.1.4 (Markov process)**.** A process $X = (X_t)_{t\geq 0}$ is a *Markov process* with respect to the filtration $\{\mathcal{F}_t\}$ if

$$\mathbb{E}[f(X_t) \mid \mathcal{F}_s] = \mathbb{E}[f(X_t) \mid X_s] \tag{2.1}$$

for all $0 \leq s \leq t$ and all bounded measurable functions f.

2.1.2 Stochastic processes

The market in practice is not deterministic, so we will mathematically construct our model with stochastic processes, indexed by time in our application.

Definition 2.1.5 (Stochastic process)**.** A *stochastic process* is a family of random variables $\{X_t : t \geq 0\}$ defined on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$.

More specifically, we will use Brownian motion to model the underlying asset price dynamics, which is a continuous-time stochastic process with continuous paths and stationary independent increments.

Definition 2.1.6 (Brownian motion)**.** A real-valued process $W = (W_t)_{t \geq 0}$ on the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ is a *Brownian motion* if it satisfies the following properties:

- $W_0 = 0$ almost surely
- The increment $W_t - W_s$ is normally distributed with mean 0 and variance $t - s$ for all $0 \leq s < t$
- The increments $W_{t_1} - W_{t_0}, \ldots, W_{t_n} - W_{t_{n-1}}$ are independent for all $0 \leq t_0 < t_1 < \cdots < t_n$
- The sample paths $t \mapsto W_t(\omega)$ are continuous

The existence of this process is established by Wiener [11] and is very powerful in modelling the randomness in the financial market.

We will now define the Itô process [12], which is a generalisation of Brownian motion, allowing for a drift term and a diffusion term, giving us more flexibility to model the asset price dynamics.

Definition 2.1.7 (Itô process)**.** Suppose $W = (W^1, \ldots, W^d)$ is a d-dimensional Brownian motion on a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$. An *Itô process* is a process $X = (X^1, \ldots, X^d)$ valued in $\mathbb{R}^d$ such that

$$X_t = X_0 + \int_0^t \mu_s \, \mathrm{d}s + \int_0^t \sigma_s \, \mathrm{d}W_s \tag{2.2}$$

where $\mu \in \mathbb{R}^d$ and $\sigma \in \mathbb{R}^{d \times d}$ are adapted processes satisfying some integrability conditions. We can also write this in differential form as

$$\mathrm{d}X_t = \mu_t \, \mathrm{d}t + \sigma_t \, \mathrm{d}W_t \tag{2.3}$$

Finally, we present Itô's lemma [12], which allows us to compute the differential of a function of an Itô process, which we will use extensively in deriving the PDEs in the option pricing problem.

Theorem 2.1.1 (Itô's lemma)**.** *Let $X = (X^1, \ldots, X^d)$ be an Itô process, with $\mathbb{T} \subseteq \mathbb{R}_{\geq 0}$ the time domain. Let $f : \mathbb{T} \times \mathbb{R}^d \to \mathbb{R} \in \mathcal{C}^{1,2}$ be a function that is once continuously differentiable in time and twice continuously differentiable in space. Then the process $f(t, X_t)$ is also an Itô process and satisfies*

$$\begin{aligned} f(t, X_t) = f(0, X_0) &+ \int_0^t \frac{\partial f}{\partial t}(s, X_s) \, \mathrm{d}s + \sum_{i=1}^{d} \int_0^t \frac{\partial f}{\partial x_i}(s, X_s) \, \mathrm{d}X_s^i \\ &+ \frac{1}{2} \sum_{i=1}^{d} \sum_{j=1}^{d} \int_0^t \frac{\partial^2 f}{\partial x_i \partial x_j}(s, X_s) \, \mathrm{d}\langle X^i, X^j \rangle_s \end{aligned} \tag{2.4}$$

2.1.3 Risk-neutral pricing

We will now formalise the financial market and establish the principle of arbitrage-free valuation that will underpin our option pricing problem. In our model, we consider a fixed time horizon $T > 0$ (our expiry) and a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t\in[0,T]}, \mathbb{P})$ where $\{\mathcal{F}_t\}$ is the filtration generated by the underlying Brownian motion. The market consists of a risk-free asset B_t that is deterministic and follows the evolution

$$B_t = \exp\left(\int_0^t r(s)\,\mathrm{d}s\right) \tag{2.5}$$

and one or more risky assets S_t that follow an Itô process.

It is important that the market is arbitrage-free, which means that there does not exist any trading strategy that can generate a positive profit with zero initial investment and zero risk. This trading strategy ϕ must be self-financing, which means that any change in the value of the portfolio is only due to changes in the assets held, without any external cash flow.

Definition 2.1.8 (Self-financing trading strategy)**.** A *self-financing trading strategy* ϕ is a predictable process such that the value of the portfolio $V_t = \phi_t^B B_t + \phi_t^S S_t$ satisfies

$$\mathrm{d}V_t = \phi_t^B \mathrm{d}B_t + \phi_t^S \mathrm{d}S_t \tag{2.6}$$

Definition 2.1.9 (Arbitrage)**.** An *arbitrage* is some self-financing trading strategy ϕ such that $V_0 = 0$, $V_T \geq 0$ almost surely and $\mathbb{P}(V_T > 0) > 0$.

By the fundamental theorem of asset pricing, the market is arbitrage-free if and only if there exists a measure $\mathbb{Q}$ such that $\mathbb{Q}$ is equivalent to $\mathbb{P}$ (they have the same null sets) and the discounted price process $\tilde{S}_t = \frac{S_t}{B_t}$ is a martingale under $\mathbb{Q}$. We will assume that our market is complete (any derivative can be replicated), and hence such a measure $\mathbb{Q}$ will be unique.

Given that the market is arbitrage-free, we can price any derivative with payoff $g(\cdot)$ at time t by taking the discounted expectation of the payoff under the risk-neutral measure $\mathbb{Q}$, giving us the risk-neutral pricing formula for a European derivative with maturity T:

$$v(t,S) = B_t \cdot \mathbb{E}^{\mathbb{Q}}\left[\frac{g(S_T)}{B_T} \;\middle|\; \mathcal{F}_t\right] \tag{2.7}$$

Throughout this project specifically, we will deal with constant interest rates, so the bond price process reduces to $B_t = e^{rt}$ where r is the constant risk-free rate. Additionally, we will also assume the Markov property (2.1.4). The arbitrage-free price of the European derivative will therefore be given by

$$v(t,S) = \mathbb{E}^{\mathbb{Q}}\left[e^{-r(T-t)} g(S_T) \;\middle|\; S_t = S\right] \tag{2.8}$$

2.1.4 Optimal stopping problem and free boundary

The American option gives the holder the right to exercise the option at any time before maturity, and hence we should identify when we should exercise the option to maximise the payoff. Therefore, we can interpret the option price as the solution to an optimal stopping problem, where we want to find the optimal *stopping time* $\tau^\star$.

Definition 2.1.10 (Stopping time)**.** A random variable $\tau : \Omega \to [0,\infty]$ is a *stopping time* if $\{\tau \leq t\} \in \mathcal{F}_t$ for all $t \geq 0$.

The price of the American option can thus be expressed as the $\mathbb{Q}$-expectation of the discounted payoff at the optimal stopping time $\tau^\star$ that maximises the expected payoff, giving the optimal stopping problem

$$v(t,S) = \sup_{\tau\in[t,T]} \mathbb{E}^{\mathbb{Q}}\left[e^{-r(\tau-t)}g(S_\tau) \,\middle|\, S_t = S\right] \tag{2.9}$$

which is simply (2.8) with the additional optimisation over the stopping time τ.

This optimal $\tau^\star$ is the first time that the option holder chooses to exercise the option, which motivates the notion of a free boundary, which is the boundary separating the exercise region and the continuation region.

Let's work with the case $t = 0$ for simplicity, but the same logic applies for any $t \in [0, T)$. For the American put, the optimal stopping problem (2.9) can be rewritten as

$$v(0,S) = \sup_{\tau\in[0,T]} \mathbb{E}\left[e^{-r\tau}(K - S_\tau)^+ \mid S_0 = S\right] \tag{2.10}$$

which is achieved by an optimal stopping time $\tau^\star$ taking the form

$$\tau^\star = \inf\{\tau \geq 0 : S_\tau \leq b^\star(\tau)\} \tag{2.11}$$

for some optimal exercise boundary $b^\star(\tau)$ [5]. This is illustrated in Figure 2.1, where we simulate one realisation of the asset price assuming a geometric Brownian motion and plot it against the exercise boundary. The illustration is based on the binomial tree solution for the American put option, which has the boundary

$$b^\star(\tau) = \sup\{S : c(\tau, S) = g(S)\} \tag{2.12}$$

which is found using binary search here, and $c(t,S)$ is the continuation value of the option with payoff $g(S)$.

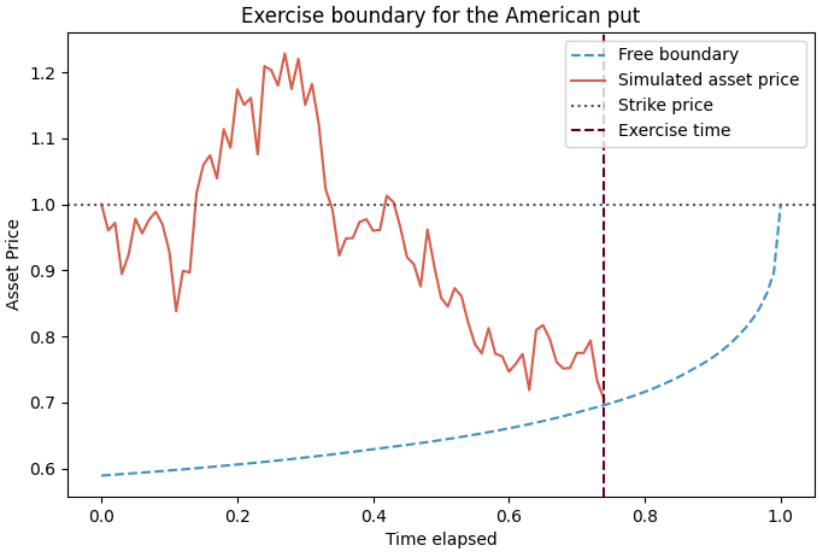

Figure 2.1 Exercise boundary for the American put (adapted from [5])

Soft classification of the free boundary

Our definition of the free boundary is analogous to a hard classification problem, where we classify points in the time-stock price space binarily to be either in the continuation region or the exercise region. However, our numerical solutions to the pricing problem are not exact, with various things such as Monte Carlo simulations and numerical differentiation introducing some level of error in our solution. Therefore, it would be more intuitive to perform a soft classification instead, where we assign probabilities to points in the time-stock price space to be in the continuation region or the exercise region, which would give us a more intuitive visualisation of the free boundary and how it evolves over time. We will do this by extending the hard classification problem. Currently, the continuation region $\mathcal{A}$ is

$$\mathcal{A} = \{(t, S) : c(t, S) > g(S)\} \cup \{(t, S) : g(S) = 0\} \tag{2.13}$$

where intuitively, the first set is those where the continuation value is greater than the early exercise value (and hence it is better to continue), and the second set is those where the payoff is zero, which are at-the-money and out-of-the-money options that are worthless under immediate exercise. We will now consider two methods to extend the definition of the continuation region to the solution of a soft classification problem.

Sigmoid soft classification

The first method is to use a sigmoid function $s : \mathbb{R} \to (0, 1)$ defined as

$$s(x) = \frac{1}{1 + e^{-x}} \tag{2.14}$$

to smoothly transition the continuation probability from 0 to 1 as the value function transitions from being smaller than the payoff to being greater than the payoff. As the sigmoid function is the CDF of the logistic distribution, we can interpret this as modelling the noise in the value function as a logistic distribution. The effect of the noise can be controlled by introducing the softness parameter $\beta > 0$, and our continuation probability function $P : \mathbb{R} \to (0, 1)$ can be defined on the relative difference from the payoff as

$$P(d) = s\left(\frac{d}{\beta}\right), \qquad d := \frac{c(t, S) - g(S)}{g(S)} \tag{2.15}$$

To tune the softness parameter, introduce some reference relative difference $\epsilon > 0$ and some target $J \in (0.5, 1)$ such that $P(\epsilon) = J$. With some simple algebraic manipulation, we can solve for β to get

$$\beta = \frac{\epsilon}{\ln\left(\frac{J}{1-J}\right)} \tag{2.16}$$

Gaussian noise soft classification

The second method is to assume Gaussianity of the noise in our estimate $\hat{c}$ of the continuation value, and that our approximated price is unbiased. In other words, we have

$$\hat{c}(t, S) = c(t, S) + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2) \tag{2.17}$$

where c is the true (latent) continuation value and σ is the standard deviation of the solution noise, which is also unknown. We can then solve the pricing problem n times to obtain the

different approximated continuation values $\hat{c}_1, \ldots, \hat{c}_n$. This allows us to use the sample mean $\bar{c}$ as our unbiased estimate of the continuation value function paired with the sample standard deviation $\hat{\sigma}$. Then the standardised statistic is distributed according to a t-distribution with $n-1$ degrees of freedom, and we can compute the continuation probability as

$$P(t,S) = \mathbb{P}_{t_{n-1}}\left(\frac{\bar{c}(t,S) - g(S)}{\hat{\sigma}(t,S)/\sqrt{n}} > 0\right) \tag{2.18}$$

which is simple as the CDF of the t-distribution is readily available.

The caveat with this method is that it requires multiple simulations of the solution, which becomes infeasible for more complex models and higher dimensions, but it gives us a more statistically grounded way to compute the continuation probability.

All that is left is to find a method to obtain the continuation value function, as it is generally latent and not directly observable, given that we predict the American option price that already takes the maximum of the continuation value and the payoff. However, we can use

$$c(t,S) = \mathbb{E}\left[e^{-rh} f(t+h, S_{t+h}) \;\middle|\; S_t = S\right] \tag{2.19}$$

where $h > 0$ is a small time step and f is the American option price function. We can obtain an unbiased estimation of this expectation by Monte Carlo: we simulate the asset price S_{t+h} at time $t+h$ given $S_t = S$ many times. For each simulation, we evaluate the option price and then take an ensemble mean for our final estimate of the continuation value function.

2.1.5 Monte Carlo integration

Monte Carlo is a powerful way to estimate integrals that may be intractable or do not have a closed-form solution, which will be useful through our project as expectations can be expressed as integrals under the probability measure. The idea is to use the law of large numbers to estimate the integral by sampling from some distribution and evaluating the integrand at the sampled points.

Suppose we want to compute the integral of some function ϕ over a domain Ω:

$$I = \int_\Omega \phi(x)\,\mathrm{d}x \tag{2.20}$$

If we are able to sample iid from some distribution with probability density function q that has positive density on Ω, then we can rewrite the integral as

$$I = \int_\Omega \frac{\phi(x)}{q(x)} q(x)\,\mathrm{d}x = \mathbb{E}\left[\frac{\phi(X)}{q(X)}\right] \tag{2.21}$$

where the expectation is taken under the distribution with density q. This expectation can be estimated by sampling $X_1, \ldots, X_N$ iid from q and taking the sample mean:

$$\hat{I} = \frac{1}{N}\sum_{j=1}^{N} \frac{\phi(X_j)}{q(X_j)} \tag{2.22}$$

This is known as importance sampling [13], and our choice of q can affect the variance of the estimator $\hat{I}$. In practice, we can choose q to be uniform over Ω, giving us the simple estimator

$$\hat{I} = \frac{1}{N}\sum_{j=1}^{N} \phi(X_j) \tag{2.23}$$

2.2 State Space Discretisation

2.2.1 Background

Recall that the option value function $v(t, S)$ is the solution to the optimal stopping problem

$$v(t, S) = \sup_{\tau \in [t,T]} \mathbb{E}\left[e^{-r(\tau - t)} g(S_\tau) \mid S_t = S\right] \tag{2.24}$$

There does not exist a closed-form solution for this problem in general, and thus we have to resort to numerical methods. We first consider methods that discretise the continuous time into a finite grid $\{t_i\}_{i=1}^N$ where we have

$$0 = t_0 < t_1 < \cdots < t_N = T \tag{2.25}$$

which we can use to index the underlying state variable S_{t_i} at time t_i. Note that the state variable S_{t_i} could depend on multiple variables, such as in the Heston model, where we have a stochastic variance term encoded in the asset price. The goal is to compute the arbitrage-free price $v(0, S_0)$ at time $t = 0$ given the initial state S_0. As we know the solution must satisfy the terminal condition $v(T, S) = g(S)$ for all S, we can use backward induction to compute the option value at each time step. This gives rise to the class of numerical methods known as tree methods.

2.2.2 Tree Methods

In a tree model, we discretise the time-state space into a finite grid, where the state variable S_{t_i} can take a finite number of values at each time step given the previous one. We will particularly explore binomial trees in this project, where each state can evolve to one of two possible states at the next time step. But to formalise the general case, let's denote S_{t_i} simply with its index S_i for convenience in this section. Suppose at time t_n we have the state variable S_n taking one of the values out of M_n possible states, that is to say $S_n \in \{S_n^1, \ldots, S_n^{M_n}\}$. Then at time t_{n+1}, each state S_n^i can evolve to the state S_{n+1}^j with the transition probability

$$p_{ij}^{(n)} = \mathbb{P}(S_{n+1} = S_{n+1}^j \mid S_n = S_n^i) \tag{2.26}$$

This is a discrete-time Markov chain, and to identify the transition probabilities, we can use the risk-neutral condition which states that the discounted price process must be a martingale under the risk-neutral measure. This gives us the condition

$$\sum_{j=1}^{M_{n+1}} p_{ij}^{(n)} S_{n+1}^j = e^{r\Delta t} S_n^i \tag{2.27}$$

where $\Delta t = t_{n+1} - t_n$ and r is the risk-free rate.

With these transition probabilities, we can compute the option value at time t_n given the option value at time t_{n+1} by using backwards induction, which gives us the recursive relation

$$v(t_n, S_n^i) = \max\left(g(S_n^i),\ e^{-r\Delta t} \sum_{j=1}^{M_{n+1}} p_{ij}^{(n)} v(t_{n+1}, S_{n+1}^j)\right) \tag{2.28}$$

where we take the maximum of the payoff and the discounted expected future price to account for early exercise for the American option. This maximum operator can easily be lifted for all

time steps for European options, or only imposed on certain time steps for Bermudan options, making the tree method very flexible. This backwards induction is applied iteratively starting from the terminal condition at time $t_N = T$ to compute the option value at time $t_0 = 0$.

An issue with tree methods is that the number of states M_n can grow exponentially with the number of time steps, leading to a very computationally expensive algorithm. To mitigate this, we use recombining trees where the states at each time step are designed to recombine with each other, so that the number of states M_n grows linearly. Figure 2.2 illustrates the difference between non-recombinant and recombinant trees. Another issue is that, while flexible, the tree method can be difficult to implement for more complex models where the state depends on multiple variables. This makes it difficult for potential extensions to price more complex derivatives on multiple assets. However, the reliability and interpretability of the tree method make it a good benchmark for simpler models.

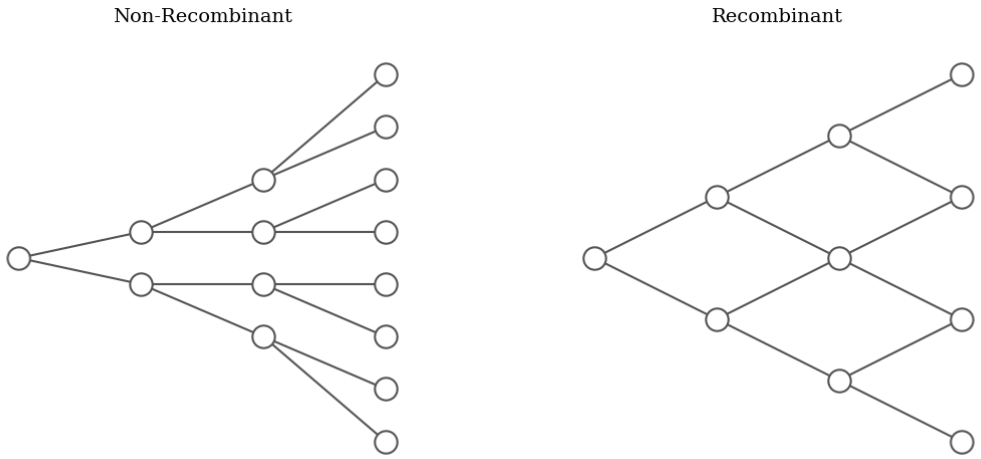

Figure 2.2 Non-recombinant trees vs recombinant trees. The number of nodes in non-recombinant trees scales exponentially with the number of time steps.

2.3 Physics-Informed Neural Networks

2.3.1 Motivation

Our option pricing problem can alternatively be formulated as a variational inequality which takes the form of a partial differential equation that is independent of the payoff $g(\cdot)$. These PDEs can be of high dimension, and are thus difficult to solve using traditional numerical methods due to the curse of dimensionality. Traditional methods such as finite difference methods require discretisation of the domain and the number of grid points (and hence algorithm complexity) scales exponentially $\mathcal{O}(N^d)$ with the dimension d of the problem.

We can attempt a neural network solution to the PDEs, as neural networks are known to be universal approximators. To be more precise, let $\Psi(\cdot)$ be a squashing activation function, which is a non-decreasing function satisfying the limits

$$\Psi(-\infty) = 0, \quad \Psi(\infty) = 1 \tag{2.29}$$

Define the class of functions $\mathscr{F}$ over a compact set $\mathcal{X}$ that can be represented by a neural network with one hidden layer, activation function Ψ and any finite number of hidden neurons. Then $\mathscr{F}$ is dense in the space of continuous functions on $\mathcal{X}$ [14]. In other words, for any

continuous target function $f^\star$ and any target accuracy $\epsilon > 0$, there exists a function $f \in \mathscr{F}$, realised by a network with sufficiently many hidden neurons, such that

$$\sup_{x \in \mathcal{X}} |f(x) - f^\star(x)| \leq \epsilon \tag{2.30}$$

Since the option price function is continuous in the time-asset price domain, we can use a neural network to find an approximation. This is favourable because the time complexity of the training process scales with the number of parameters, which does not scale exponentially. To do this, we will use the framework of *physics-informed neural networks* (PINNs) to solve the PDEs, which have been shown to be effective for solving high-dimensional PDEs [6], by avoiding mesh construction and leveraging the interpolation capability of neural networks [15]. The loss function of the neural network is designed to penalise deviations from the PDE as well as deviations from the boundary conditions. By training the neural network to minimise this loss function, we can obtain an approximation to the solution of the PDE. This method has been applied to option pricing problems in the literature, such as in [16] for the Black-Scholes model, but we will look to apply it to more complex models such as the Heston model as well as to options on multiple assets.

2.3.2 Methodology

We first provide a general setup for PINNs, before modifying it to tailor our specific problem of option pricing. Formally, suppose we have an unknown latent solution $u(t, x)$ which is governed by a PDE of the form

$$\frac{\partial u}{\partial t} + \mathcal{L}[u] = 0, \qquad t \in [0, T], x \in \Omega \tag{2.31}$$

where $\mathcal{L}$ is a (not necessarily linear) differential operator. Suppose additionally that this PDE is subject to the boundary conditions

$$u(0, x) = g(x), \qquad x \in \Omega \tag{2.32}$$

$$\mathcal{B}[u] = 0, \qquad t \in [0, T], x \in \partial\Omega \tag{2.33}$$

where $\mathcal{B}$ is a boundary operator. Then the unknown solution u can be approximated by a deep neural network $f_\theta(t, x)$ where θ are the parameters of the neural network, trained by minimising a loss function composed of deviations from the PDE residual, boundary conditions and initial conditions [17]:

$$L(\theta) = \lambda_r L_r(\theta) + \lambda_{bc} L_{bc}(\theta) + \lambda_{ic} L_{ic}(\theta) \tag{2.34}$$

where the loss components are mathematically defined as

$$L_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \mathcal{R}_\theta(t_r^i, x_r^i) \right|^2 \tag{2.35}$$

$$L_{bc}(\theta) = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \left| \mathcal{B}[f_\theta](t_{bc}^i, x_{bc}^i) \right|^2 \tag{2.36}$$

$$L_{ic}(\theta) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left| f_\theta(0, x_{ic}^i) - g(x_{ic}^i) \right|^2 \tag{2.37}$$

where the PDE residual $\mathcal{R}_\theta$ is defined as

$$\mathcal{R}_\theta(t, x) = \frac{\partial f_\theta}{\partial t}(t, x) + \mathcal{L}[f_\theta](t, x) \tag{2.38}$$

We additionally use the sampled points $\{x_{ic}^i\}_{i=1}^{N_{ic}}$ from the initial condition domain, $\{(t_{bc}^i, x_{bc}^i)\}_{i=1}^{N_{bc}}$ from the boundary condition domain and $\{(t_r^i, x_r^i)\}_{i=1}^{N_r}$ from the PDE domain, sampled from their respective domains (whether it be uniformly or according to a specific distribution) at each iteration of the gradient descent algorithm.

There are different methods to choose the loss weights $\lambda_r, \lambda_{bc}, \lambda_{ic}$. We could simply fix the weights to be constant throughout training, or use adaptive weights that change during training, which we will discuss in Section 2.3.3.

For our specific problem of option pricing, we can modify the above setup to accommodate our variational inequality formulation. The PDE to solve is dependent on the model for the asset dynamics, and the boundary conditions are given by the payoff function of the option. In general, we will have to solve a problem of the form

$$\min\left\{-\frac{\partial u}{\partial t} + \mathcal{L}[u],\ u - g\right\} = 0, \qquad t \in [0,T], x \in \Omega \tag{2.39}$$

$$u(T,x) = g(x), \qquad x \in \Omega \tag{2.40}$$

$$\mathcal{B}_k[u] = 0 \qquad t \in [0,T], x \in \partial\Omega, k = 1,\dots,K \tag{2.41}$$

where k indexes the K different boundary conditions. This is very similar to the general setup for PINNs, except that we have a variational inequality instead of a PDE, and we have a terminal condition instead of an initial condition. However, we can still use the framework by noting we can make the substitution $t \mapsto T - t$ to remove the sign of the negative partial derivative in the PDE term and to convert the terminal condition to an initial condition. The variational inequality can be handled by modifying the PDE residual loss (2.38) to

$$\mathcal{R}_\theta(t,x) = \min\left\{\frac{\partial f_\theta}{\partial t}(t,x) + \mathcal{L}[f_\theta](t,x),\ f_\theta(t,x) - g(x)\right\} \tag{2.42}$$

With everything in place, we can train the neural network with gradient descent:

$$\theta_{n+1} = \theta_n - \eta_n \nabla_\theta L(\theta_n) \tag{2.43}$$

where η_n is the learning rate at iteration n, either kept constant or decayed over time with a schedule.

2.3.3 Adaptive Loss Weighting

While keeping the loss weights $\lambda_r, \lambda_{bc}, \lambda_{ic}$ fixed is simple, it can be difficult to tune correctly and can lead to suboptimal performance. Ideally, we would like to balance the different loss components so that they are of the same order of magnitude, which can be achieved by using adaptive loss weighting methods, as proposed in [17]. The idea is to evaluate the contribution of each loss component on the total loss and adjust the weights accordingly to balance their contributions. In practice, we will use a moving average to prevent the weights from changing too rapidly, leading to instability in training. Mathematically, we compute the adaptive weights

$$\hat{\lambda}_r = \frac{\|\nabla_\theta L_r(\theta)\| + \|\nabla_\theta L_{bc}(\theta)\| + \|\nabla_\theta L_{ic}(\theta)\|}{\|\nabla_\theta L_r(\theta)\|}$$

$$\hat{\lambda}_{bc} = \frac{\|\nabla_\theta L_r(\theta)\| + \|\nabla_\theta L_{bc}(\theta)\| + \|\nabla_\theta L_{ic}(\theta)\|}{\|\nabla_\theta L_{bc}(\theta)\|}$$

$$\hat{\lambda}_{ic} = \frac{\|\nabla_\theta L_r(\theta)\| + \|\nabla_\theta L_{bc}(\theta)\| + \|\nabla_\theta L_{ic}(\theta)\|}{\|\nabla_\theta L_{ic}(\theta)\|}$$

and update the global weights $\lambda = (\lambda_r, \lambda_{bc}, \lambda_{ic})$ with a smoothness $\alpha \in (0, 1)$ as a moving average

$$\lambda_{\text{new}} = \alpha \lambda_{\text{old}} + (1 - \alpha)\hat{\lambda}_{\text{new}} \tag{2.44}$$

This is not performed at each iteration, but rather at a fixed frequency to prevent the weights from changing too rapidly. The literature [17] suggests updating the weights every 1000 iterations with $\alpha = 0.9$ in general.

2.3.4 Numerical Optimisation

Gradient Descent

Our neural network will learn through gradient descent, in particular Adam, which uses adaptive learning rates and momentum to speed up convergence. Note this does not conflict with the adaptive loss weighting described in Section 2.3.3, as the adaptive loss weighting is used to balance the different loss components, while Adam is used to adapt the learning rate for each parameter based on the historical gradients.

We will additionally impose a schedule to decay the learning rate over time, which starts at a conventional value of 10^{-3} to allow for fast learning at the beginning, and decays by a factor every certain number of iterations, allowing for finer convergence in later iterations. These are also tuned by observing the training curve, and can vary for different models and different payoffs.

Early Stopping

Early stopping is a regularisation technique mainly used to prevent overfitting in neural network training. However, we are training on a new random training set each epoch, so we do not need to worry about overfitting, and early stopping is implemented to avoid unnecessary training for efficiency.

The idea of early stopping is to stop the training process once we do not observe a significant improvement in the loss function after a certain number of iterations, which we call the patience. This can be tuned by observing the training curve and finding a good balance between training time and performance. Once the patience is exceeded without observing a significant improvement, the training is stopped and the model reverted to the best performing model observed during training. The loss used to evaluate the improvement is computed on a validation set that is held through the training process, a set of points in the time-state space that is not used for training but only for evaluation.

Throughout this project, we will set the improvement threshold to be 10^{-6} or 10^{-7} and the patience to be between 500 and 1500 iterations depending on the complexity of the model.

Fine-tuning

The Adam optimiser is a first-order method that uses the first moment of the gradients to adapt the learning rate, so we do not leverage the information given by the second moment of the gradients. It is particularly good for navigating the loss landscape at the beginning of training, but in the later stages, it can exhibit poor convergence behaviour. This is because its stochastic noise and momentum can prevent it from tight convergence, requiring a very small learning

rate to prevent oscillation. [18] explains in detail the shortcomings of Adam, where they construct explicit counterexamples of simple convex functions where Adam fails to converge to the optimal solution.

For PINNs, this is a problem, as we require a very small residual loss to satisfy the PDE accurately. We will therefore use a second-order method to fine-tune the model: Adam would serve as the global exploration to get us close to a minimum, and then we can switch to a second-order method to converge tightly. This makes us turn to quasi-Newton methods such as the BFGS algorithm [19], which maintains a dense Hessian matrix which is updated at each iteration, and a step size is computed with a line search method, which is more effective than setting a learning rate. However, this can be computationally expensive for large models, and the literature suggests we fine-tune with L-BFGS [20] after having already completed gradient descent with the Adam optimiser. L-BFGS is a limited-memory version of BFGS that only stores a few vectors to approximate the Hessian, which is much more efficient for large models, at the trade-off of a shorter memory of the curvature information across iterations [21].

Chapter 3

Black-Scholes Model

The Black-Scholes model [1] is the fundamental model for option pricing, making the assumption that the underlying asset price follows a geometric Brownian motion. As with the market introduced in Section 2.1.3, the market consists of two underlying assets:

- The risk-free asset, described by a deterministic function

$$\mathrm{d}B_t = rB_t\mathrm{d}t \tag{3.1}$$

  where for convenience, we set $B_0 = 1$ and r is the constant risk-free interest rate with continuous compounding. This has the solution $B_t = e^{rt}$ for $t \geq 0$.

- The risky asset S_t, which has the price process of a geometric Brownian motion (GBM), which satisfies the stochastic differential equation

$$\mathrm{d}S_t = \mu S_t\mathrm{d}t + \sigma S_t\mathrm{d}W_t \qquad 0 \leq t \leq T \tag{3.2}$$

  for a given drift μ and volatility σ. We can solve this SDE to yield the unique solution

$$S_t = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma W_t\right) \qquad 0 \leq t \leq T \tag{3.3}$$

For convenience, we will refer to the risk-free asset as the *bond*, and the risky asset as the *asset* for the remainder of this chapter.

Under risk-neutral pricing, we can find an equivalent probability measure $\mathbb{Q}$ under which the discounted asset price is a martingale, and hence all assets earn the risk-free rate in expectation after discounting. Under the risk-neutral measure, the SDE for the asset becomes

$$\mathrm{d}S_t = rS_t\mathrm{d}t + \sigma S_t\mathrm{d}W_t^{\mathbb{Q}} \tag{3.4}$$

where the drift has been replaced by the risk-free rate r. The derivation is done via Girsanov's theorem, and details can be found in Chapter 5 of [22].

3.1 Closed-form solution for European options

Our first benchmark will be the closed-form solution for the European option, which would be used to evaluate the accuracy of the binomial tree later. The derivation of this formula is widely

available and hence we will only include the main ideas. Using the risk-neutral pricing formula, the value of the European call option with strike K and expiry T is given by

$$v(0, S_0) = e^{-rT}\mathbb{E}^{\mathbb{Q}}\big[(S_T - K)^+ \mid S_0\big] \tag{3.5}$$

We use the solution of the geometric Brownian motion (3.3) to note that

$$S_T = S_0 \exp\left((r - \tfrac{1}{2}\sigma^2)T + \sigma W_T^{\mathbb{Q}}\right) \tag{3.6}$$

under the risk-neutral measure $\mathbb{Q}$, having replaced the drift μ with the risk-free rate r. The Brownian motion $W_T^{\mathbb{Q}}$ has zero mean and variance T, so it can be written as $\sqrt{T}Z$, where we have the standard normal variable $Z \sim \mathcal{N}(0, 1)$. Therefore, S_T becomes

$$S_T = S_0 \exp\left((r - \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z\right) \tag{3.7}$$

This can be plugged into the risk-neutral pricing formula (3.5): the exercise event $\{S_T > K\}$ is equivalent to $\{Z > -d_2\}$ where

$$d_2 = \frac{\log(S_0/K) + (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}} \tag{3.8}$$

The expectation (3.5) can be split, where we denote $\mathbf{1}_A$ the indicator variable for the event A:

$$\begin{aligned} v(0, S_0) &= e^{-rT}\mathbb{E}^{\mathbb{Q}}[(S_T - K)^+] && (3.9)\\ &= e^{-rT}\mathbb{E}^{\mathbb{Q}}[(S_T - K)\mathbf{1}_{\{S_T > K\}}] && (3.10)\\ &= e^{-rT}\mathbb{E}^{\mathbb{Q}}[S_T\mathbf{1}_{\{S_T > K\}}] - Ke^{-rT}\mathbb{Q}(S_T > K) && (3.11) \end{aligned}$$

The second probability is immediate as we see $\mathbb{Q}(S_T > K) = \mathbb{Q}(Z > -d_2) = \Phi(d_2)$ by symmetry, where Φ is the CDF of the standard normal. We can complete the square in the Gaussian integrand for the first expectation to get the final formula [1], for $d_1 = d_2 + \sigma\sqrt{T}$:

$$v(0, S_0) = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{3.12}$$

With the call option price, we can also obtain the put option price via put-call parity, which states that

$$v_{\text{call}}(0, S_0) - v_{\text{put}}(0, S_0) = S_0 - Ke^{-rT} \tag{3.13}$$

3.2 Binomial Tree Method

We can use the binomial discrete-time model to price both European and American options with the tree method. This model uses the simplification that the asset price can only move to one of two possible new prices in a time step. To avoid explosion in the number of nodes, we will only consider recombinant trees so that the algorithm performs with quadratic time complexity (look back at Figure 2.2 for an illustration of recombinant trees). We will verify the approximated solution's validity against the closed-form formula for European options, and use its American variant as a benchmark for later methods.

To formalise the tree construction, let's fix a time horizon $T > 0$. For some depth $n \geq 1$, divide the interval $[0, T]$ into n even steps each of length $\Delta t = T/n$. Our asset S_i, where i is a discrete time index representing the time $i\Delta t$, will follow a multiplicative binomial evolution:

$$S_{i+1} = \begin{cases} S_i u & \text{with probability } p \\ S_i d & \text{with probability } 1 - p \end{cases} \qquad S_0 > 0 \tag{3.14}$$

where u and d are the model parameters determining the two factors of the asset price change per time step.

As with (3.1), the financial market will also consist of a risk-free asset B_t, discretised so that it satisfies

$$B_{i+1} = B_i e^{r\Delta t}, \qquad B_0 = 1 \tag{3.15}$$

where r is the continuously-compounded risk-free rate, typically considered interest. For an arbitrage free market, we should additionally ensure that

$$d < e^{r\Delta t} < u \tag{3.16}$$

for sufficiently small Δt, else we can make a riskless profit by shorting the asset and buying the bond, or vice versa.

To employ risk-neutral pricing, we additionally require the discounted asset price to have zero drift (admitting a martingale measure). More precisely, with the bond as numeraire, we have the discounted price $\tilde{S}_i$ at time step i given by

$$\tilde{S}_i = \frac{S_i}{B_i} = e^{-ri\Delta t} S_i \tag{3.17}$$

We require some probability measure $\mathbb{Q}$, equivalent to the physical measure $\mathbb{P}$ (having the same non-zero support) such that

$$\mathbb{E}^{\mathbb{Q}}[\tilde{S}_{i+1} \mid \mathcal{F}_i] = \tilde{S}_i \tag{3.18}$$

where $\mathcal{F}_i$ is the filtration at time step i. To calculate the risk-neutral probability, we can apply this to our model:

$$e^{-r(i+1)\Delta t}\big[pS_i u + (1-p)S_i d\big] = e^{-ri\Delta t} S_i \tag{3.19}$$

Collect the p terms and cancel out the discount factor to get

$$p(S_i u - S_i d) + S_i d = e^{r\Delta t} S_i \tag{3.20}$$

Divide both sides by S_i and rearrange to make p the subject to obtain

$$p = \frac{e^{r\Delta t} - d}{u - d} \tag{3.21}$$

This is what we will use in our binomial model.

To price the option at time 0, we will perform backward induction from expiry, since we know that the price is just the payoff $g : \mathbb{R}^+ \to \mathbb{R}$ as the option expires. As the discounted price process is a martingale under the risk-free measure $\mathbb{Q}$, the arbitrage-free value of the derivative at time $i\Delta t$ is necessarily

$$v(t = i\Delta t) = \underbrace{e^{-r(T-i\Delta t)}}_{\text{discount}} \mathbb{E}^{\mathbb{Q}}[g(S_T) \mid \mathcal{F}_i] \tag{3.22}$$

We will utilise the Cox-Ross-Rubinstein (CRR) model [3] for our implementation, which chooses the parameters

$$u = e^{\sigma\sqrt{\Delta t}} \qquad d = e^{-\sigma\sqrt{\Delta t}} \tag{3.23}$$

satisfying the arbitrage free condition, and normalising the local variance of returns to the volatility parameter σ^2. They also conveniently multiply to 1.

To implement this, we need to first compute the price tree. This would be, for the price $S_{i,j}$ at time step i after j up-moves (so clearly $j \leq i$)

$$S_{i,j} = S_0 u^j d^{i-j} \tag{3.24}$$

As the payoff is known at maturity, we can simply set, with $v_{i,j}$ the value of the option at time step i with j up-moves:

$$v_{n,j} = g(S_{n,j}) \qquad 0 \leq j \leq n \tag{3.25}$$

and recursively set

$$v_{i,j} = e^{-r\Delta t}\big(p \cdot v_{i+1,j+1} + (1-p) \cdot v_{i+1,j}\big) \tag{3.26}$$

for $0 \leq j \leq i$. This would be it for European options, but since early exercise is allowed, we will compare this value with the payoff of the price at step i with j up-moves, which is just $g(S_{i,j})$. Thus the value is given by

$$v_{i,j} = \max\left(e^{-r\Delta t}\big(pv_{i+1,j+1} + (1-p)v_{i+1,j}\big),\ g(S_{i,j})\right) \tag{3.27}$$

The value at the root node $v_{0,0}$ would be the arbitrage-free option price.

3.3 Physics-Informed Neural Networks

In order to use the framework of PINNs as detailed in Section 2.3, we need to first derive the PDE satisfied by the option price, as well as the boundary conditions. We will first work with the one-dimensional case, then discuss the extension to the multi-dimensional case and how we could build a benchmark against which we evaluate our results.

3.3.1 PINN formulation for one-dimensional case

In order to set up our PINN, we need to first derive the PDE satisfied by the option price. Recall the value function $v : [0,T] \times \mathbb{R}^+ \to \mathbb{R}$ of an American put with payoff $g : \mathbb{R}^+ \to \mathbb{R}$ at time t with asset price $S_t = S$ is given by

$$v(t,S) = \sup_{\tau \in [t,T]} \mathbb{E}\big[e^{-r(\tau-t)} g(S_\tau) \mid S_t = S\big] \tag{3.28}$$

where τ is any stopping time in $[t,T]$ and for convenience we will simply denote $\mathbb{E}_t$ as the conditional expectation at time t, given $S_t = S$ in this section. For an American option, we always have two options:

- Immediately exercise the option and get payoff $g(S)$. Since this is not necessarily optimal, the payoff must bound the option value from below:

$$v(t,S) \geq g(S) \tag{3.29}$$

- Do nothing and let the option continue to time $t+h$ with $h > 0$ a very small number. By using the supremum definition of $v(t,S)$ (3.28) and swapping the expectation and supremum [23], we arrive at

$$e^{rh} v(t,S) \geq \mathbb{E}_t\big[v(t+h, S_{t+h})\big] \tag{3.30}$$

  and the inequality turns into an equality at the limit $h \to 0$.

We will additionally assume that $v \in \mathcal{C}^{1,2}$ (the value function is continuously differentiable with respect to t and twice continuously differentiable with respect to S) so that we can apply Itô's formula as in Theorem 2.1.1. Let's model the asset price S as a geometric Brownian motion satisfying under the risk-neutral measure

$$\mathrm{d}S_t = rS_t\mathrm{d}t + \sigma S_t \mathrm{d}W_t \tag{3.31}$$

Then by Itô's lemma and taking expectations of both sides we have

$$\mathbb{E}_t\big[v(t+h, S_{t+h})\big] = v(t,S) + \mathbb{E}_t\left[\int_t^{t+h} \left(\frac{\partial v}{\partial t}(u, S_u) + \frac{\partial v}{\partial S}(u, S_u) rS_u + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(u, S_u)\sigma^2 S_u^2\right) \mathrm{d}u\right] \tag{3.32}$$

Now let's substitute (3.30) into the LHS of (3.32), which gives us

$$e^{rh} v(t,S) \geq v(t,S) + \mathbb{E}_t\left[\int_t^{t+h} \left(\frac{\partial v}{\partial t}(u, S_u) + \frac{\partial v}{\partial S}(u, S_u) rS_u + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(u, S_u)\sigma^2 S_u^2\right) \mathrm{d}u\right] \tag{3.33}$$

We can use the Taylor approximation $e^{rh} = 1 + rh + o(h)$, which allows us to cancel the $v(t,S)$ term on both sides. Dividing both sides by h in the same step, we get

$$rv(t,S) + \frac{o(h)}{h} \geq \mathbb{E}_t\left[\frac{1}{h}\int_t^{t+h} \left(\frac{\partial v}{\partial t}(u, S_u) + \frac{\partial v}{\partial S}(u, S_u) rS_u + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(u, S_u)\sigma^2 S_u^2\right) \mathrm{d}u\right] \tag{3.34}$$

Sending $h \to 0$ makes the higher order term $o(h)/h$ vanish, and the expectation of the integral would collapse to the integrand. Thus we obtain

$$rv(t,S) \geq \frac{\partial v}{\partial t}(t,S) + \frac{\partial v}{\partial S}(t,S) rS + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(t,S)\sigma^2 S^2 \tag{3.35}$$

or alternatively, we define the linear parabolic operator $\mathcal{L}$ [8]:

$$\mathcal{L}[v] = rv(t,S) - \frac{\partial v}{\partial S}(t,S) rS - \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(t,S)\sigma^2 S^2 \tag{3.36}$$

Thus we can concisely express this inequality as

$$\min\left(-\frac{\partial v}{\partial t}(t,S) + \mathcal{L}[v], v(t,S) - g(S)\right) \geq 0 \tag{3.37}$$

However, note that at time t, we must either exercise the option or do nothing and continue. One of these two options must be optimal, implying that one of the arguments must be 0. Thus we actually have an equality:

$$\min\left(-\frac{\partial v}{\partial t}(t,S) + \mathcal{L}[v], v(t,S) - g(S)\right) = 0 \tag{3.38}$$

which is the variational equation [24] for the American option. Note this is exactly the formulation of the PINN problem (2.39).

We are now ready to derive the boundary conditions. Our option has a terminal payoff at time T given by $g(S)$, so we have the terminal condition

$$v(T,S) = g(S) \tag{3.39}$$

which satisfies the form of the terminal condition for PINNs (2.40). For the spatial boundary conditions, we have two boundaries to consider: $S \to 0$ and $S \to \infty$. In practice, we will have

to truncate the domain to $\Omega = (S_{\text{min}}, S_{\text{max}})$ for some sufficiently small S_{min} and sufficiently large S_{max} for training. At the lower boundary, we have the boundary condition

$$v(t, S_{\text{min}}) = g(S_{\text{min}}) \tag{3.40}$$

because it is always optimal to exercise the option when the asset is at its lowest modelled price. At the upper boundary, we assume the option is too far out of the money to ever finish in the money, and hence the option price is approximately 0. Thus we have the boundary condition

$$v(t, S_{\text{max}}) = 0 \tag{3.41}$$

These three conditions together would form the boundary setup (2.41) for our PINN.

3.3.2 Multi-asset case

We will now extend our PINN framework to the multi-dimensional case, where we have n underlying assets, and hence the neural network f will take $n + 1$ inputs (time and the price of each asset). The variational inequality can be derived with a similar argument as in the one-dimensional case, but we will use the multi-dimensional case of Itô's lemma (Theorem 2.1.1). In order to be able to evaluate the accuracy of our results, we will consider a specific multi-asset payoff such that there exists an equivalent one-dimensional reduction. This allows us to use the binomial tree method (or indeed the one-dimensional PINN solution) as a benchmark, giving us confidence for the PINN solution on more complex payoffs.

In the multi-dimensional case, we have n underlying assets with price processes following a correlated geometric Brownian motion. Mathematically, let each asset S^i satisfy under the risk-free measure $\mathbb{Q}$, for $i = 1, \ldots, n$,

$$\mathrm{d}S_t^i = rS_t^i \mathrm{d}t + \sigma_i S_t^i \mathrm{d}W_t^i \tag{3.42}$$

with quadratic covariation $\mathrm{d}\langle W^i, W^j \rangle_t = \rho_{ij} \mathrm{d}t$ for all i, j pairs.

3.3.3 Reduction of multi-asset to one-asset case

We will work with a specific payoff function that allows us to reduce the multi-dimensional case to a one-dimensional case, which would, in turn, give us confidence that our PINN implementation is reliable, and we can change the payoff function to more complex ones and be reasonably assured that the results are still correct.

As the geometric Brownian motion is multiplicative, for our benchmark, we will consider the put on the product of the assets. More precisely, we consider the put option with payoff

$$g(S^1, \ldots, S^n) = \left(K - \prod_{i=1}^{n} S^i \right)^+ \tag{3.43}$$

To represent the dynamics of the product of the assets, we can apply Itô's formula to the log of the price process. The log representation of the Itô process (refer to Definition 2.1.7) is

$$\ln S_t^i = \ln S_0^i + \int_0^t \left(r - \frac{1}{2}\sigma_i^2 \right) \mathrm{d}s + \int_0^t \sigma_i \, \mathrm{d}W_s^i \tag{3.44}$$

Define the new asset X to be

$$X_t := \prod_{i=1}^{n} S_t^i \tag{3.45}$$

and hence we have

$$\ln X_t = \sum_{i=1}^{n} \ln S_t^i = \ln X_0 + \int_0^t \left(nr - \frac{1}{2} \sum_{i=1}^{n} \sigma_i^2 \right) \mathrm{d}s + \sum_{i=1}^{n} \int_0^t \sigma_i \, \mathrm{d}W_s^i \tag{3.46}$$

By taking the differential we have

$$\mathrm{d} \ln X_t = \left(nr - \frac{1}{2} \sum_{i=1}^{n} \sigma_i^2 \right) \mathrm{d}t + \sum_{i=1}^{n} \sigma_i \, \mathrm{d}W_t^i \tag{3.47}$$

The quadratic variation of the martingale term is

$$\mathrm{d} \left\langle \sum_{i=1}^{n} \sigma_i W^i, \sum_{j=1}^{n} \sigma_j W^j \right\rangle_t = \sum_{i=1}^{n} \sum_{j=1}^{n} \sigma_i \sigma_j \rho_{ij} \mathrm{d}t =: \sigma_X^2 \mathrm{d}t \tag{3.48}$$

i.e. $\sigma_X^2 = \sum_{i=1}^{n} \sum_{j=1}^{n} \sigma_i \sigma_j \rho_{ij}$. So we can express the second term of (3.47) in terms of a new Brownian motion $\widetilde{W}$:

$$\sum_{i=1}^{n} \sigma_i \, \mathrm{d}W_t^i = \sigma_X \mathrm{d}\widetilde{W}_t \tag{3.49}$$

Going back, we have

$$\mathrm{d} \ln X_t = \left(nr - \frac{1}{2} \sum_{i=1}^{n} \sigma_i^2 \right) \mathrm{d}t + \sigma_X \mathrm{d}\widetilde{W}_t \tag{3.50}$$

Exponentiating using Itô's formula gets us

$$\mathrm{d}X_t = X_t \left[\left(nr - \frac{1}{2} \sum_{i=1}^{n} \sigma_i^2 \right) \mathrm{d}t + \sigma_X \mathrm{d}\widetilde{W}_t \right] + \frac{1}{2} X_t \sigma_X^2 \mathrm{d}t \tag{3.51}$$

which we can rearrange to get

$$\mathrm{d}X_t = \left(nr - \frac{1}{2} \sum_{i=1}^{n} \sigma_i^2 + \frac{1}{2} \sigma_X^2 \right) X_t \mathrm{d}t + \sigma_X X_t \, \mathrm{d}\widetilde{W}_t \tag{3.52}$$

which is a geometric Brownian motion with volatility σ_X and drift r_X where

$$r_X = nr - \frac{1}{2} \sum_{i=1}^{n} \sigma_i^2 + \frac{1}{2} \sigma_X^2 \tag{3.53}$$

$$= nr + \sum_{i<j} \sigma_i \sigma_j \rho_{ij} \tag{3.54}$$

by matching coefficients. We can verify the equivalence by simulating many price paths for the assets and comparing their product against the simulations of price paths for the equivalent asset. Using the initial values (chosen arbitrarily)

$$r = 0.05 \qquad S_0 = (10, 5, 3)^\top \qquad \sigma = (0.2, 0.3, 0.1)^\top \qquad \Sigma = \begin{pmatrix} 1 & 0.6 & -0.2 \\ 0.6 & 1 & 0.4 \\ -0.2 & 0.4 & 1 \end{pmatrix} \tag{3.55}$$

where Σ is the correlation matrix, we can plot the histogram of final prices in Figure 3.1.

They look very similar, and we can further statistically test if they come from the same distribution. Using Kolmogorov-Smirnov's 2-sample test on the null hypothesis that they come from the same distribution, we obtain a p-value of 0.8491, which does not provide sufficient evidence to reject the null hypothesis, which is consistent with the two distributions being equal.

Therefore, we will be able to price the put option on the product of multiple assets and compare that to the price of the put option on this single constructed asset as a benchmark.

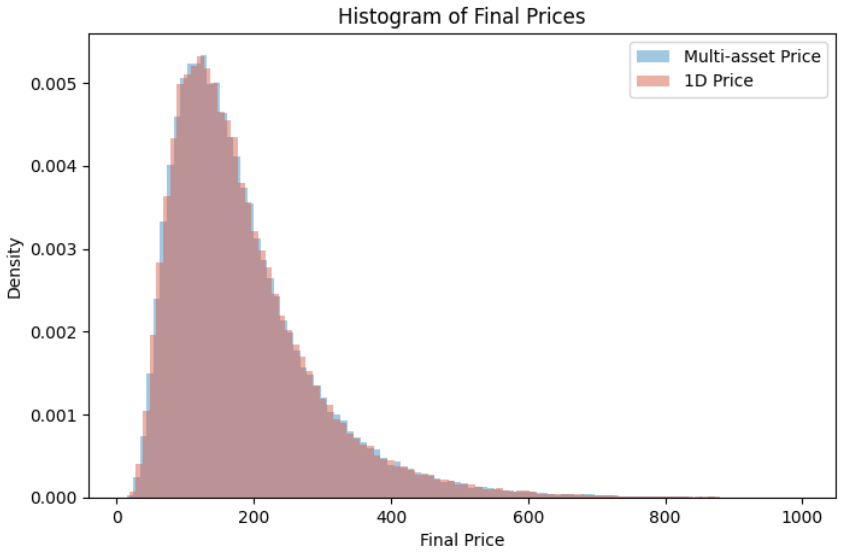

Figure 3.1 Histogram of final prices for product of assets and equivalent asset

3.3.4 PINN formulation for multi-dimensional case

With the benchmark in place, we now need to derive the variational equation for the multi-dimensional case where we have n assets. This is a straightforward extension of the one-dimensional case, where the value function $v(t, S^1, \dots, S^n)$ now takes $n+1$ inputs. For notation, we will express the multiple assets as a vector $S = (S^1, \dots, S^n)^\top$, so the value function can be written more concisely as $v(t, S)$. Similarly, the payoff function will be written as $g(S^1, \dots, S^n) = g(S)$.

The value function stays

$$v(t,S) = \sup_{\tau \in [t,T]} \mathbb{E}\left[e^{-r(\tau - t)} g(S_\tau) \mid S_t = S\right] \tag{3.56}$$

Should we choose not to exercise the option at time t, we can apply the same argument as in the one-dimensional case: considering a small $h > 0$, we can apply Itô's lemma on multiple dimensions to yield

$$\begin{aligned} v(t+h, S_{t+h}) = v(t,S) + \int_t^{t+h} \Bigg(& \frac{\partial v}{\partial t}(u, S_u) + \sum_{i=1}^n \frac{\partial v}{\partial S^i}(u, S_u) r S_u^i \\ & + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 v}{\partial S^i \partial S^j}(u, S_u) \sigma_i \sigma_j \rho_{ij} S_u^i S_u^j \Bigg) \, \mathrm{d}u \end{aligned} \tag{3.57}$$

The steps from here are identical to the one-dimensional case, where we can take expectations of both sides and use the second order Taylor expansion of the exponential to get the variational inequality

$$r v(t,S) \geq \frac{\partial v}{\partial t}(t,S) + \sum_{i=1}^n \frac{\partial v}{\partial S^i}(t,S) r S^i + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 v}{\partial S^i \partial S^j}(t,S) \sigma_i \sigma_j \rho_{ij} S^i S^j \tag{3.58}$$

where to simplify into the PINN framework as in (2.39), we can define the operator $\mathcal{L}$ as

$$\mathcal{L}[v] = r v(t,S) - \sum_{i=1}^n \frac{\partial v}{\partial S^i}(t,S) r S^i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 v}{\partial S^i \partial S^j}(t,S) \sigma_i \sigma_j \rho_{ij} S^i S^j \tag{3.59}$$

and the problem reduces to the same variational inequality as in the one-dimensional case (3.38), with a different operator $\mathcal{L}$.

To finish the PINN formulation, we need to derive the boundary conditions. The terminal condition is straightforward, as it is given by the payoff function:

$$v(T,S) = g(S) = \left(K - \prod_{i=1}^{n} S^i\right)^+ \tag{3.60}$$

For the spatial boundary conditions, we will again truncate the domain to

$$\Omega = \left\{S = (S^1, \ldots, S^n) \in \mathbb{R}^n : S^i \in (S^i_{\text{min}}, S^i_{\text{max}}) \text{ for } i = 1, \ldots, n\right\} \tag{3.61}$$

for some sufficiently small and large respective minimum and maximum values for each asset. The lower boundary will be sampled from the set

$$\partial_{\text{lower}}\Omega = \left\{S \in \Omega : S^i = S^i_{\text{min}} \text{ for some } i\right\} \tag{3.62}$$

where we have the boundary condition

$$v(t,S) = g(S), \qquad S \in \partial_{\text{lower}}\Omega \tag{3.63}$$

and the upper boundary will be sampled from the set

$$\partial_{\text{upper}}\Omega = \left\{S \in \Omega : S^i = S^i_{\text{max}} \text{ for some } i\right\} \tag{3.64}$$

with the boundary condition

$$v(t,S) = 0, \qquad S \in \partial_{\text{upper}}\Omega \tag{3.65}$$

analogous to the one-dimensional case. This is a simplification because we can argue this is not the case for mixed corners and points on the upper face where the other assets are small, but it works well in practice and much easier to implement.

Of course, this framework could be easily adapted to other payoff functions, where the boundary conditions would change accordingly (but not the variational inequality). This will be explored later in the numerical results when we consider a different payoff. However, for our benchmark, the put on the product of assets is the setup we will use.

3.4 Sobolev Deep Neural Networks

For the Black-Scholes model, we will additionally explore a different method to price American puts with neural networks, but this time instead of the traditional boundary conditions, we will work with fractional Sobolev norms to construct our loss function. This is because it has been shown that conditional on the loss converging to 0, the neural network will converge to the true solution [8], which is a stronger guarantee than just pointwise convergence. Additionally, other than the terminal condition, we will not need to explicitly specify the boundary conditions. This makes the method more flexible and more applicable to a wider range of problems, which is a desirable property for a general method.

The theory of Sobolev spaces is rich and we will not attempt to cover all the details, to avoid diverting attention from the main focus of the project. However, we make particular use of fractional Sobolev spaces, which are a generalisation of the integer order Sobolev spaces. These definitions are taken from [25, 26] which are very good resources for the underlying theory.

3.4.1 Sobolev spaces

Throughout this section, let $\Omega \subset \mathbb{R}^n$ be an open bounded set with Lipschitz boundary, on which our target function is defined, and let $p \in [1, +\infty)$ be the integrability parameter. Recall the Lebesgue space

$$L^p(\Omega) := \left\{ u : \Omega \to \mathbb{R} : u \text{ is measurable and } \int_\Omega |u(x)|^p \, \mathrm{d}x < \infty \right\} \tag{3.66}$$

and the integer order Sobolev space $W^{m,p}(\Omega)$ for $m \in \mathbb{N}$ [25],

$$W^{m,p}(\Omega) := \{ u \in L^p(\Omega) : D^\alpha u \in L^p(\Omega),\ |\alpha| \leq m \}, \tag{3.67}$$

where $D^\alpha u$ denotes the weak derivative of u with respect to the multi-index α.

The fractional Sobolev space $W^{s,p}(\Omega)$ is defined with the fractional exponent $s \in (0, 1)$ as [26]:

$$W^{s,p}(\Omega) := \left\{ u \in L^p(\Omega) : \frac{|u(x) - u(y)|}{|x - y|^{n/p+s}} \in L^p(\Omega \times \Omega) \right\}, \tag{3.68}$$

which can be interpreted as an intermediate Banach space between $L^p(\Omega)$ and $W^{1,p}(\Omega)$, endowed with the norm

$$\begin{aligned} \|u\|^p_{W^{s,p}(\Omega)} &= \|u\|^p_{L^p(\Omega)} + [u]^p_{W^{s,p}(\Omega)} \\ &= \int_\Omega |u|^p \, \mathrm{d}x + \int_\Omega \int_\Omega \frac{|u(x) - u(y)|^p}{|x - y|^{n+sp}} \, \mathrm{d}x \, \mathrm{d}y, \end{aligned} \tag{3.69}$$

where we have used the *Gagliardo seminorm* of u:

$$[u]_{W^{s,p}(\Omega)} := \left(\int_\Omega \int_\Omega \frac{|u(x) - u(y)|^p}{|x - y|^{n+sp}} \, \mathrm{d}x \, \mathrm{d}y \right)^{1/p}. \tag{3.70}$$

We can extend this definition to non-integer $s > 1$. In the decomposition $s = m + \sigma$ with $m \in \mathbb{N}$ and $\sigma \in (0, 1)$, we define

$$W^{s,p}(\Omega) := \{ u \in W^{m,p}(\Omega) : D^\alpha u \in W^{\sigma,p}(\Omega),\ |\alpha| = m \} \tag{3.71}$$

endowed with the natural norm

$$\|u\|^p_{W^{s,p}(\Omega)} := \|u\|^p_{W^{m,p}(\Omega)} + \sum_{|\alpha|=m} [D^\alpha u]^p_{W^{\sigma,p}(\Omega)}. \tag{3.72}$$

In the special case $p = 2$, the fractional Sobolev space becomes a Hilbert space, which we denote $H^s(\Omega) := W^{s,2}(\Omega)$. This is the specific case we will be working with in our loss construction.

3.4.2 Model

Suppose we would like to price an option on n assets $S = (S^1, \ldots, S^n)$. As asset prices are theoretically unbounded, we truncate the domain in our model much like the vanilla PINN. Thus the domain of the target function can be chosen to be the open set

$$\Omega = (S^1_{\text{min}}, S^1_{\text{max}}) \times \cdots \times (S^n_{\text{min}}, S^n_{\text{max}}) \tag{3.73}$$

In our study we will make the simplification that $S^i_{\text{min}} = S^j_{\text{min}}$ and $S^i_{\text{max}} = S^j_{\text{max}}$ for all pairs i, j, which we will denote with a and b respectively (but of course this method still works for more general cases). Our simplified spaces are therefore

$$\Omega = (a,b)^n \qquad \Omega_T = (0,T) \times \Omega \qquad \Sigma = (0,T) \times \Gamma \tag{3.74}$$

where $\Gamma := \partial\Omega$ is the boundary of Ω, a hyper-rectangle. We will use the representation

$$\Gamma = \bigcup_{i=1}^{n} \left(\{x \in [a,b]^n : x_i = a\} \cup \{x \in [a,b]^n : x_i = b\} \right) \tag{3.75}$$

which is not the simplest representation, but it is the most useful for our purposes as it is decomposed into faces, needed for the normal derivative. Note that in the one-dimensional case, the boundary Γ reduces simply to the endpoints $\Gamma = \{a, b\}$.

3.4.3 Loss function

Our loss function would be a weighted sum of four loss components $\sum_{i=1}^{4} \lambda_i J_i$, each of which ensures regulation of a specific aspect of our function, with the weights either kept constant or chosen through adaptive loss weighting as presented in Section 2.3.3. We present the generalised loss construction, applicable for any dimension.

1. The variational loss: this is as before, controlling the accuracy of our function in the interior of our domain:
$$J_1 = \| \min\{\mathcal{G}[f], f - g\} \|^2_{L^2(\Omega_T)} \tag{3.76}$$

2. The energy loss: before we were only controlling the L^2 norm of the payoff, but we require the option value to equal the payoff as a function and not just pointwise. Thus we need to incorporate the gradient into the loss so that the spatial slope of the payoff is respected.
$$J_2 = \| f(T,\cdot) - g(\cdot) \|^2_{H^1(\Omega)} \tag{3.77}$$
where we have the norm defined as
$$\|d\|^2_{H^1(\Omega)} = \|d\|^2_{L^2(\Omega)} + \|\nabla d\|^2_{L^2(\Omega)} \tag{3.78}$$
and the derivative
$$\nabla d = \left(\partial_{x_1} d, \ldots, \partial_{x_n} d \right) \tag{3.79}$$

3. The Dirichlet trace loss [27]: this enforces $f = g$ on the spatial boundary Σ. We have
$$J_3 = \| f(t,x) - g(x) \|^2_{H^{\frac{3}{4},\frac{3}{2}}(\Sigma)} \tag{3.80}$$
where we select the fractional Sobolev space $H^{\frac{3}{4},\frac{3}{2}}(\Sigma)$ from [28], and the fractional norm is defined as
$$\begin{aligned} \|d\|^2_{H^{\frac{3}{4},\frac{3}{2}}(\Sigma)} = \|d\|^2_{H^{0,1}(\Sigma)} &+ \int_0^T \int_0^T \frac{\|d(t_1,\cdot) - d(t_2,\cdot)\|^2_{L^2(\Gamma)}}{|t_1 - t_2|^{2.5}} \, \mathrm{d}t_1 \, \mathrm{d}t_2 \\ &+ \int_0^T \int_\Gamma \int_\Gamma \frac{|d(t,x) - d(t,y)|^2}{\|x - y\|^n} \, \mathrm{d}\sigma(x) \mathrm{d}\sigma(y) \mathrm{d}t \end{aligned} \tag{3.81}$$
The double time integral penalises the variation of f between two time points, and the double space integral penalises the variation of f between two spatial points.

The time exponent 2.5 is equal to $2p + 1$ where p is the decimal part of the time fractional exponent, and the space exponent is equal to $2\theta + n - 1$ where θ is the decimal part of the space fractional exponent, and n the dimension. Note that here $\mathrm{d}\sigma(x)$ and $\mathrm{d}\sigma(y)$ indicate the surface measures on Γ, but in practice we can treat these as $\mathrm{d}x$ and $\mathrm{d}y$.

4. The Neumann trace loss [27]: this ensures the normal derivative of $f - g$ vanishes on the boundary Σ. We have

$$J_4 = \|\nabla(f-g)\cdot\nu\|^2_{H^{\frac{1}{4},\frac{1}{2}}(\Sigma)} \tag{3.82}$$

where our space $H^{\frac{1}{4},\frac{1}{2}}(\Sigma)$ is again selected from [28], with ν the unit normal vector pointing outwards from the domain Ω. That is to say from an implementation perspective, the normal derivative is $\nabla(f-g)\cdot\nu = \partial_{x_{i^\star}}(f-g)$ with $x_{i^\star}$ the coordinate normal to the face of the boundary we are on, justifying the choice of the representation of Γ as a union of faces. Rigorously, we require the sign, but in practice this is not necessary, as we will take the square in the end. The fractional norm is thus written

$$\begin{aligned}\|d\|^2_{H^{\frac{1}{4},\frac{1}{2}}(\Sigma)} = \|d\|^2_{L^2(\Sigma)} &+ \int_0^T\int_0^T \frac{\|d(t_1,\cdot)-d(t_2,\cdot)\|^2_{L^2(\Gamma)}}{|t_1-t_2|^{1.5}}\,\mathrm{d}t_1\mathrm{d}t_2 \\ &+ \int_0^T\int_\Gamma\int_\Gamma \frac{|d(t,x)-d(t,y)|^2}{\|x-y\|^n}\,\mathrm{d}\sigma(x)\mathrm{d}\sigma(y)\mathrm{d}t\end{aligned} \tag{3.83}$$

because $H^{0,0} = L^2$.

The third and fourth losses are those that enforce f to solve the option pricing variational inequality with regularity in the time and space dimensions beyond pointwise [29]. In practice, we will compute a Monte Carlo estimate of these integrals by sampling pairs (t_1, t_2) from the domain, taking care that they are not too close to each other by rejecting pairs that have a distance less than a threshold. We then evaluate the integrand and take the mean across all pairs.

3.5 Numerical Results

3.5.1 Tree benchmark against closed-form solution

First we will evaluate the performance of the benchmark binomial tree method for the one-dimensional case against the closed-form solution, which we would take as the ground truth. To do this, we will compute the tree option price without taking the maximum of the payoff and continuation value at each step so that we can compare the tree price against the Black-Scholes price.

First, we will evaluate the tree's accuracy on the most sensitive point, which is at time $t = 0$ and when the initial asset price S is equal to the strike price K (at the money). Figure 3.2 shows the convergence of the tree price to the closed-form Black-Scholes price for European put options as we increase the depth of the tree. We can see that around 1000 steps is what we need to yield a sufficiently close price to the closed-form solution, and increasing the depth further only yields us diminishing improvements, so 1000 is the depth we will use for our tree benchmark.

Evaluating the tree price against the closed-form solution for different initial asset prices S at the initial time $t = 0$ in Figure 3.3, we can see that the tree price is very close to the ground truth, with less than 0.05% relative error until we are far out of the money where the option price is very close to 0 and unsurprisingly yields greater errors, where machine precision becomes a limiting factor.

We can see that the tree price also closely tracks the term structure of the option price, as shown in Figure 3.4, where the error is again less than 0.05%, getting even closer to zero as we get closer to maturity, which is expected as the tree has not done as many backward induction steps to get to the initial time, where errors can accumulate.

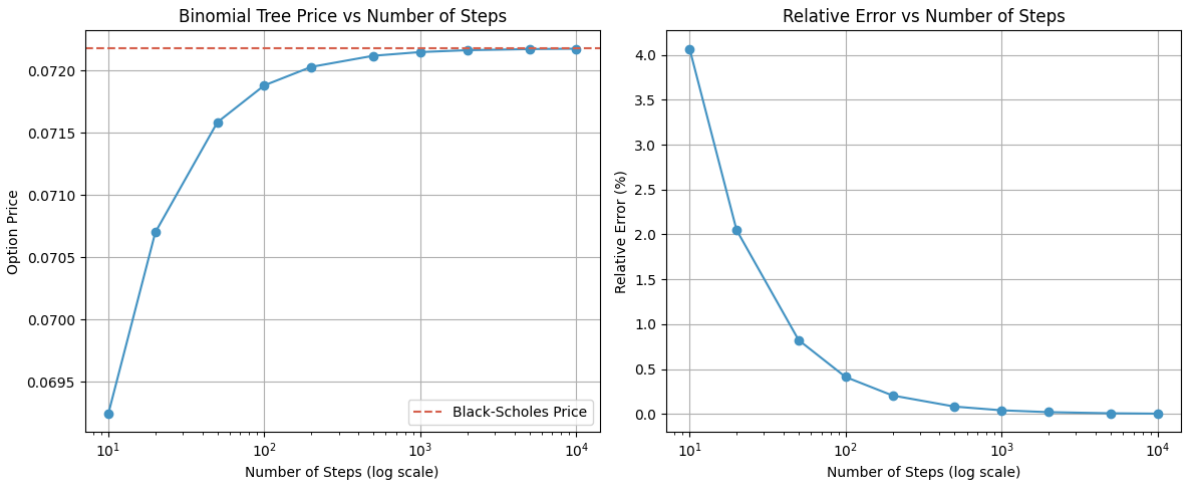

Figure 3.2 Tree convergence at $t = 0, S = K$ over different depths

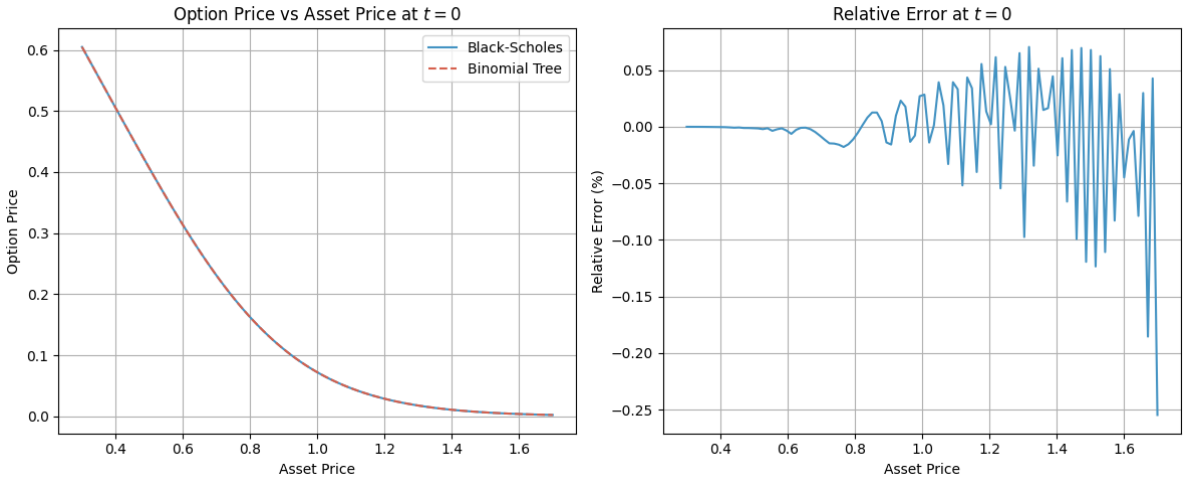

Figure 3.3 Comparison of tree and closed-form prices for European put option with varying initial asset price

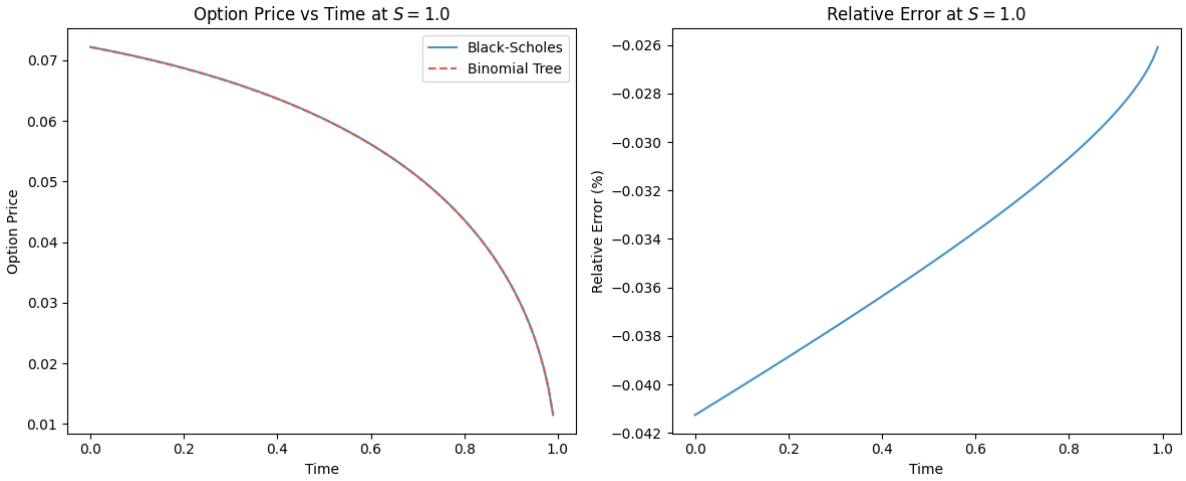

Figure 3.4 Comparison of tree and closed-form prices for European put option with varying time

Overall, we can conclude that the tree benchmark yields very strong results against the closed-form solution, giving us good confidence that it will provide accurate results for American options too, where we do not have a closed-form solution to compare against. This will serve as our benchmark for evaluating the performance of our PINN implementation for American options.

3.5.2 PINN results against tree benchmark

We will now train our PINN on the one-dimensional case and compare the results against the tree benchmark on the American put on the asset with parameters $K = 1, r = 0.1, \sigma = 0.3, T = 1$ across a range of possible initial asset prices $S_0 \in [0, 3]$. This will be done through a feedforward neural network with 3 hidden layers each of width 32, with learning rate 0.001, batch size 10000 with 30000 maximum epochs with early stopping patience of 500 epochs. We train 25 models with different random initialisation and compute the ensemble mean for our final price estimate.

Figure 3.5 shows the convergence of the loss function for one run, and we can see that the loss converges well, with each of them having similar magnitudes, converging in the order of 10^{-5}.

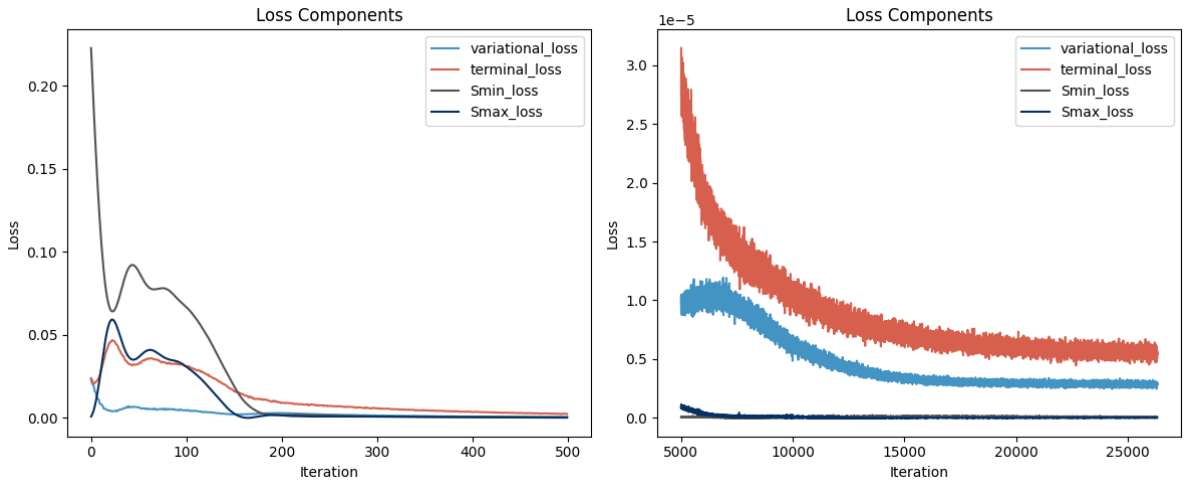

Figure 3.5 Loss convergence for the one-dimensional PINN for one run

Figure 3.6 shows the heatmap of the PINN price against the tree price across the domain of time and initial asset price. We can see that the PINN price is very close to the tree price across the domain, where the time value is correctly captured at the most sensitive point of $S = K$ and $t = 0$ as seen in Figure 3.7. This figure additionally shows the model capturing the term structure of the option price, where the PINN price closely tracks the tree price across time for $S = K$.

We can also zoom in on the most sensitive point of $(t, S) = (0, K)$ and evaluate the error. Table 3.1 shows the mean PINN price across 25 runs with its 95% confidence interval, and we can see that the relative error is around 3%, which is good considering we were sampling across the whole domain and not just at this point. However, we can see that the benchmark tree price lies

Tree price	PINN price	Relative error
0.0833	0.0860 ± 0.0006	3.33%

Table 3.1 PINN price at the money and initial time against tree benchmark

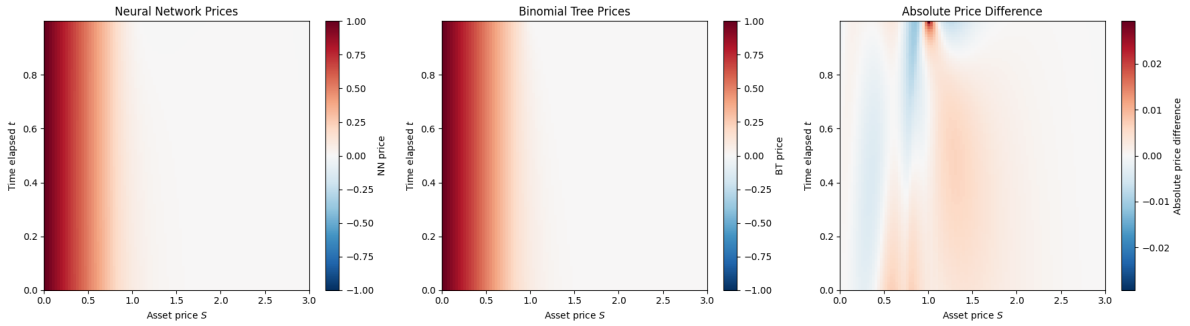

Figure 3.6 PINN vs tree price comparison heatmap across domain

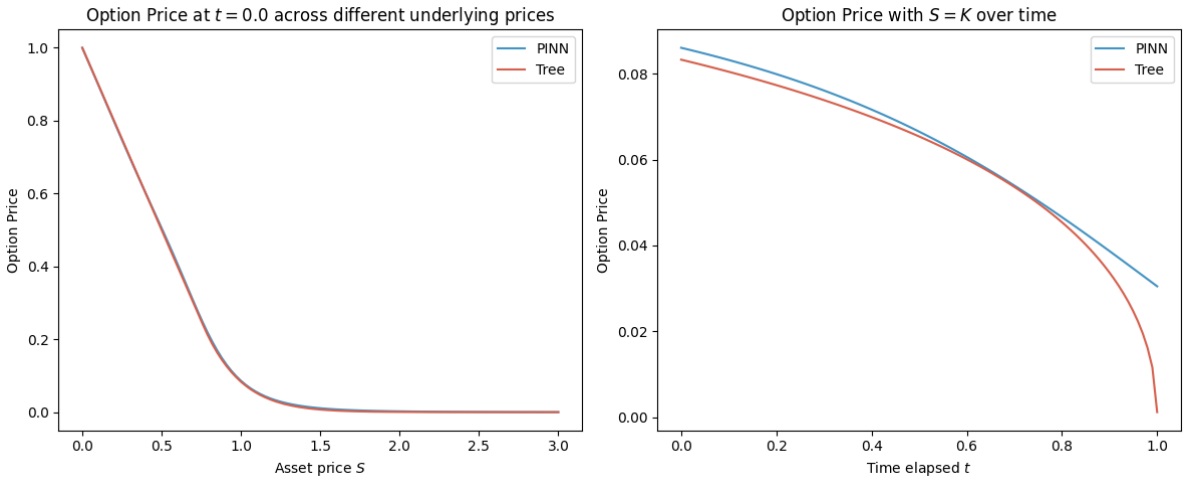

Figure 3.7 Price comparison for varying underlying price, and for term structure at $S = K$

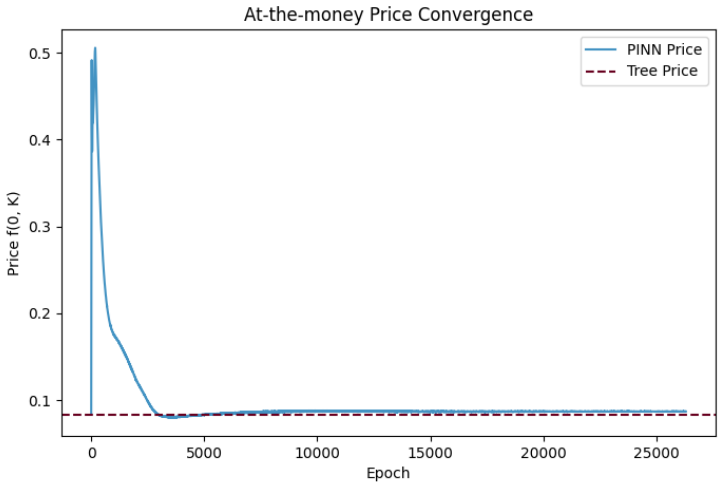

Figure 3.8 $f(0, K)$ across training epochs for one run

around 9 standard deviations away from the PINN mean price, suggesting that there is some significant bias in the PINN approximation.

We can investigate this apparent bias by tracking the price $f(t = 0, S = K)$ across different training epochs, as shown in Figure 3.8. We can see that the price spikes above the tree price and then settles around the marked benchmark tree price, seemingly converged at a value slightly greater than the tree price. This confirms that there is some bias in the PINN approximation that prevents it from getting closer to the tree price, which we are taking to be the ground truth.

3.5.3 Multi-dimensional PINN

We move on to training a PINN on multiple assets. For simplicity and visualisation purposes, we will train a two-dimensional PINN on the American put with the product of two assets as the payoff. We choose arbitrarily the parameters $K = 1, r = 0.1, T = 1$ with asset volatilities $\sigma = (0.2, 0.3)^\top$ and correlation between assets $\rho = 0.5$. Both of these assets will have a truncated interior domain (so $\Omega = (0, 3)^2$) and we will sample uniformly from the whole domain for training. The model architecture will be similar to the one-dimensional case, with a bigger feedforward neural network of 4 hidden layers each of width 64, learning rate 0.001 and batch size 10000. This will be trained for a maximum of 40000 epochs with early stopping patience of 500 epochs across 10 different seeds. Figure 3.9 shows the heatmap of the PINN price against the tree price, where we take the S^1-S^2 cross section and fix time at $t = 0$. We can see that the PINN price is close to the tree price across the domain, with some stronger bias at the in-the-money regions where the price is higher and hence the absolute error is not as significant when we take relative error into consideration.

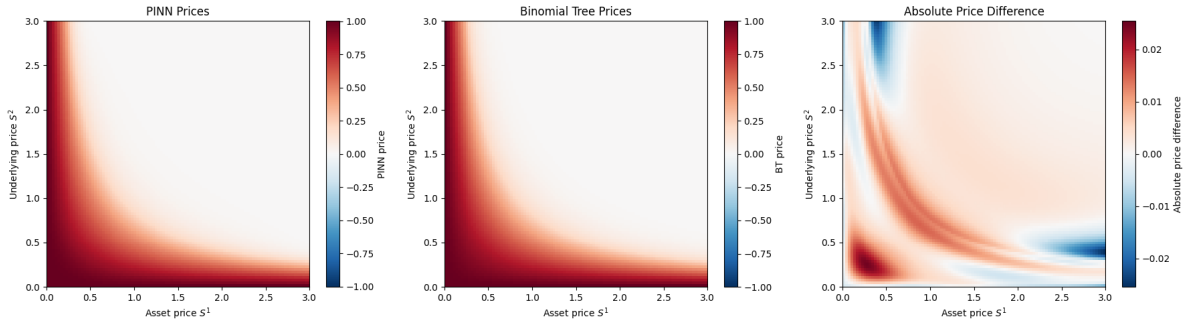

Figure 3.9 PINN vs tree price comparison heatmap across S^1-S^2 cross section at $t = 0$

We can zoom in on the behaviour along the line $S^1 \times S^2 = K$, and plot the price along this line across time, shown in the left figure of Figure 3.10. We can see that the PINN fit is satisfactory, with inconsistencies at the tails due to our boundary conditions, which are set at $S = 0$ and $S = 3$ for both assets, which are not the true boundaries of the problem and are instead a model simplification we made (as mentioned when constructing the loss, we neglect the mixed corners of the domain and certain parts of the upper face for simplicity).

Overall, by Table 3.2, we can see that the relative error at the most sensitive point at the triple $(0, \sqrt{K}, \sqrt{K})$ is around 7% which is considerably higher than the one-dimensional case, which is expected as we are now sampling across a much larger domain, and we are constrained by

Tree price	PINN price	Relative error
0.0993	0.1060 ± 0.0048	6.75%

Table 3.2 Prices $f(0, \sqrt{K}, \sqrt{K})$ for the two-dimensional PINN against tree benchmark

model capacity and training time due to personal hardware constraints, which is a common issue for multi-dimensional PINNs.

We can also take the cross section at $S^1 = S^2$ to evaluate the term structure of the option price, as shown in the right figure of Figure 3.10. We can see that the PINN has managed to capture the theta decay of the option price, but the consistent bias of the price is evident here and the model struggles to capture the more rapid theta decay as we approach maturity. This may be due to the fact that gamma approaches a Dirac delta function at the money as we approach maturity, whose sharp feature is difficult for the model to capture when our samples are sparser in the expanded domain.

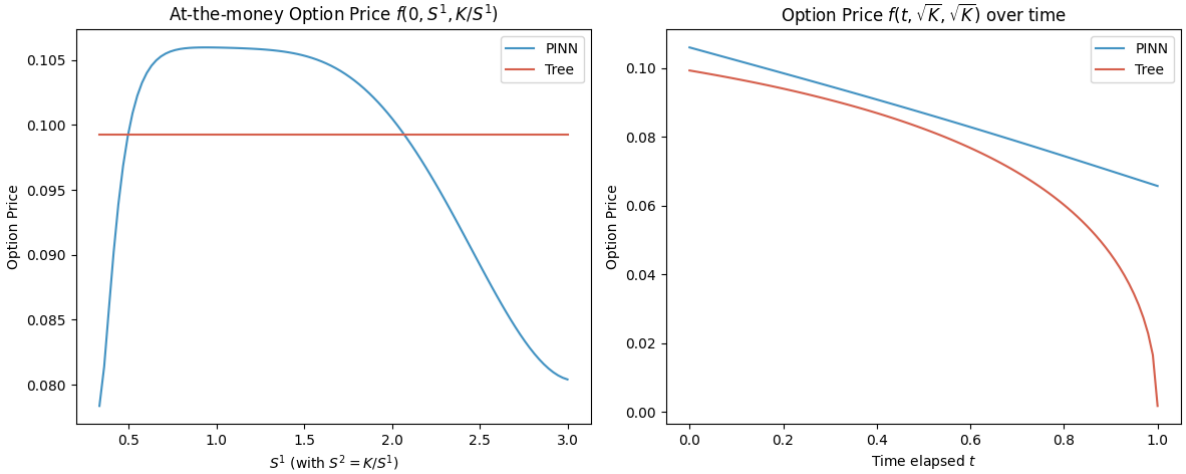

Figure 3.10 PINN vs tree price comparison for at the money cross section and term structure

3.5.4 Sobolev PINN results

One-dimensional case

We will now train a PINN with the Sobolev loss function, with the same model architecture and training setup as the vanilla PINN, and compare the results against the tree benchmark. We can see from Figure 3.11 that the Sobolev PINN price also gets close to the tree price across the domain, but the inaccuracy here is more significant than the vanilla PINN, especially at the $(t = 0, S = K)$ point, which is arguably the point we are most interested in. However, considering that with the exception of the terminal condition, the boundary conditions of the loss function aren't manufactured from the payoff function directly, we can see that this method could be

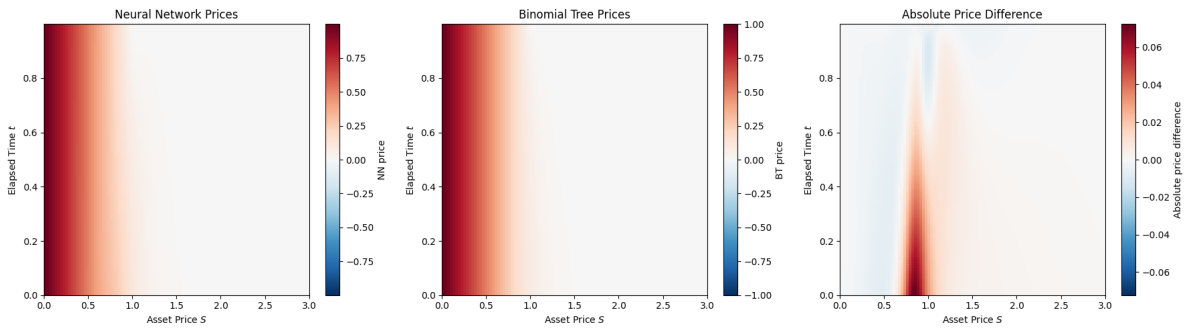

Figure 3.11 Sobolev PINN vs tree price comparison heatmap across domain

effective for more general problems where boundary conditions are not as straightforward to implement, and the price is still in the right ballpark.

We can see especially in Figure 3.12 that the Sobolev PINN manages to capture the term structure of the option price with the theta collapse as we approach maturity, and the error is more significant due to a hump in the price against asset price around the strike price. This suggests that the model is capable of a good fit across the domain, but may be undertrained or have too small a capacity.

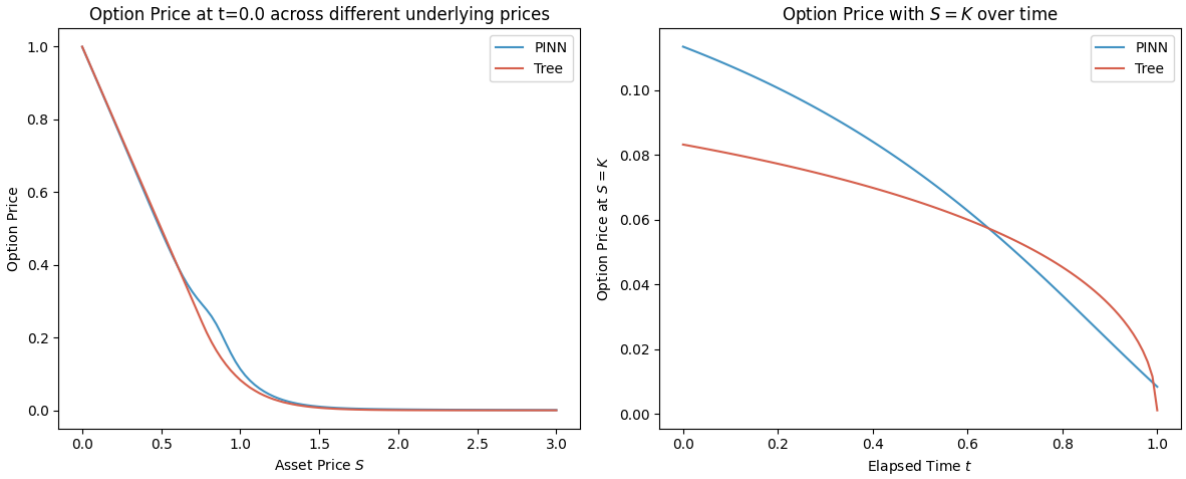

Figure 3.12 Price comparison for varying underlying price, and for term structure at $S = K$ with the Sobolev PINN

Overall this method is promising, but it may require more careful training and tuning to get better results as presented in [8], who demonstrated its effectiveness through extensive training on the GPU with much larger networks, which is beyond our scope.

Multi-dimensional case

We will now present our results for the multi-dimensional case trained by the Sobolev network, where we have added an L-BFGS optimiser after having trained with Adam for a good number of epochs for a marginal improvement in the prices, where the validation loss decreases from 3.6×10^{-3} after Adam to 1.2×10^{-4} after the fine-tuning phase, a 30x improvement.

Figure 3.13 shows the heatmap of the Sobolev PINN price against the tree price across the S^1-S^2

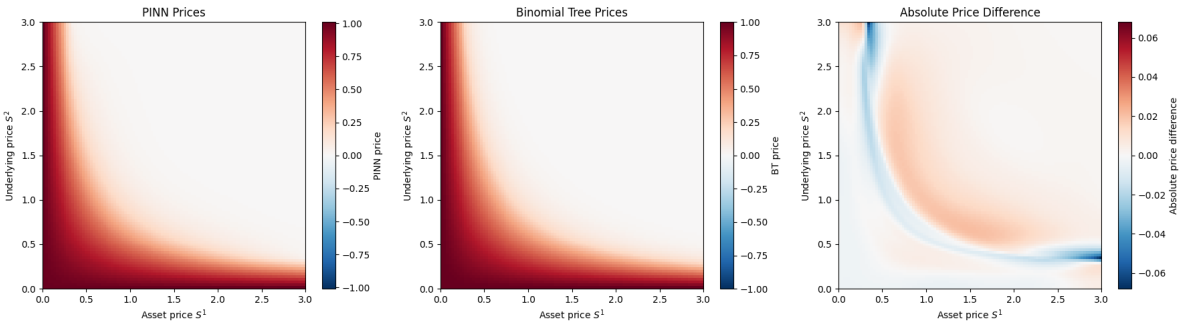

Figure 3.13 Sobolev PINN vs tree price comparison heatmap across S^1-S^2 cross section at $t = 0$

cross section at $t = 0$. We can see that the Sobolev PINN price has captured the correct shape of the price across the domain, with greater inaccuracy than the vanilla PINN.

We can see that the bias is the strongest on the at-the-money line, which we can examine more closely in the left figure of Figure 3.14. We see a similar behaviour as the vanilla case exhibited here, where the PINN overprices for more moderate values of S^1 and underprices for more extreme values of S^1. This brings the case of training an enlarged domain into question, where we only evaluate our model on a smaller interior of the domain.

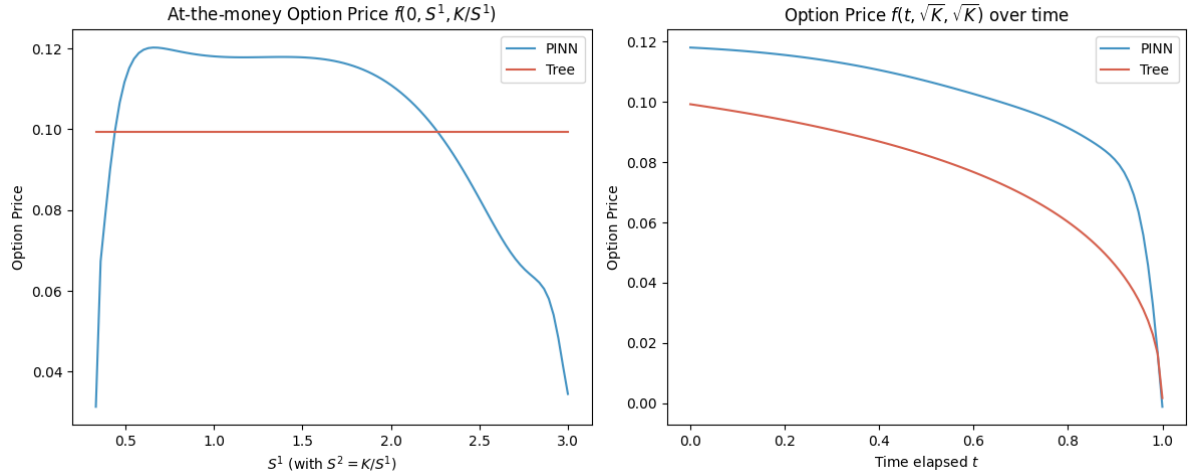

Figure 3.14 Price comparison for varying underlying price, and for term structure at $S = K$ with the Sobolev PINN

The right figure of Figure 3.14 shows the term structure of the option price, with the asset prices fixed at $S^1 = S^2 = \sqrt{K}$. We can see that the Sobolev PINN captures the theta decay of the option price better than the vanilla PINN, but there is a systematic overpricing bias across the whole time dimension. We have attempted to combat this by noticing that the errors accumulate backwards from expiry $t = T$ to the initial time $t = 0$, so we have attempted to train the model with causal weighting instead of a uniform average. This is where we put more weight on the later time steps until the residual loss is sufficiently small, and then relax this weighting gradually to a uniform average. This has improved the accuracy from over 100% relative error to around 20% relative error for the option price at $t = 0$. A relative error of this size is not ideal, but there is still a significant improvement nonetheless with the addition of this technique.

Even though this approach yielded lower accuracy than that of the vanilla PINN, we could still see its strengths over the vanilla PINN (such as it being capable of capturing the collapse of the option price to zero as we approach maturity). With more careful training and tuning, the accuracy could definitely be improved, which is an avenue for future work. We will prefer the vanilla PINN for the free boundary calculation due to its better accuracy, but it was interesting nonetheless to see how we could construct the loss function from another angle altogether, from a regularity perspective, without relying on manual boundary conditions.

3.5.5 Free boundary

One-dimensional case

For the one-dimensional free boundary, we will first evaluate the free boundary from the tree benchmark. Here, the continuation values are extracted through a small modification to the tree

algorithm, where we compute the price of the Bermudan option with the same parameters with possibility of exercise at every time step except for at $t = 0$, effectively serving as a continuation value of the American option at $t = 0$. This continuation value is compared with the intrinsic value (the payoff) like we introduced in equation (2.12) in the mathematical background. The left figure in Figure 3.15 shows the continuation probabilities across different (t, S) pairs, and here we use hard classification as binomial tree pricing is deterministic.

We can see that the free boundary is correctly captured at a glance, where we exercise the option when the asset price is sufficiently low, whose threshold increases as we get closer to maturity, consistent with the intuition that the option is losing time value.

We now evaluate the free boundary from the PINN. The central figure in Figure 3.15 shows the continuation probabilities across different (t, S) pairs using the sigmoid function as described in equation (2.15). We can see that the shape of the free boundary is similar to the tree benchmark, and with soft classification the lowest probability is around 0.4, which is not unintuitive as in the exercise region, the continuation value is theoretically equal to the payoff, and hence we should be indifferent between exercising and continuing.

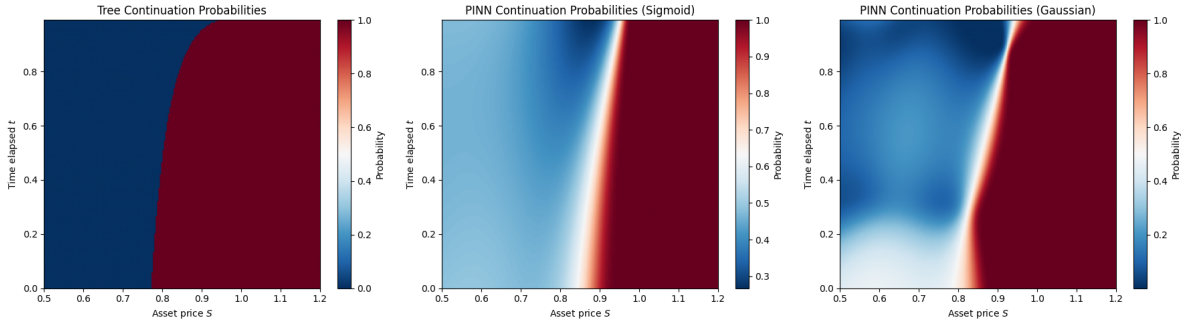

Figure 3.15 Estimated continuation probabilities from PINN for the American put option

To use the Gaussian method, we will first evaluate its validity by plotting the distribution of the standardised price estimates across all seeds in Figure 3.16, noting that as the variance is unknown and we use the sample variance, the standardised price estimates would follow a t-distribution with $n - 1$ degrees of freedom where n is the number of seeds.

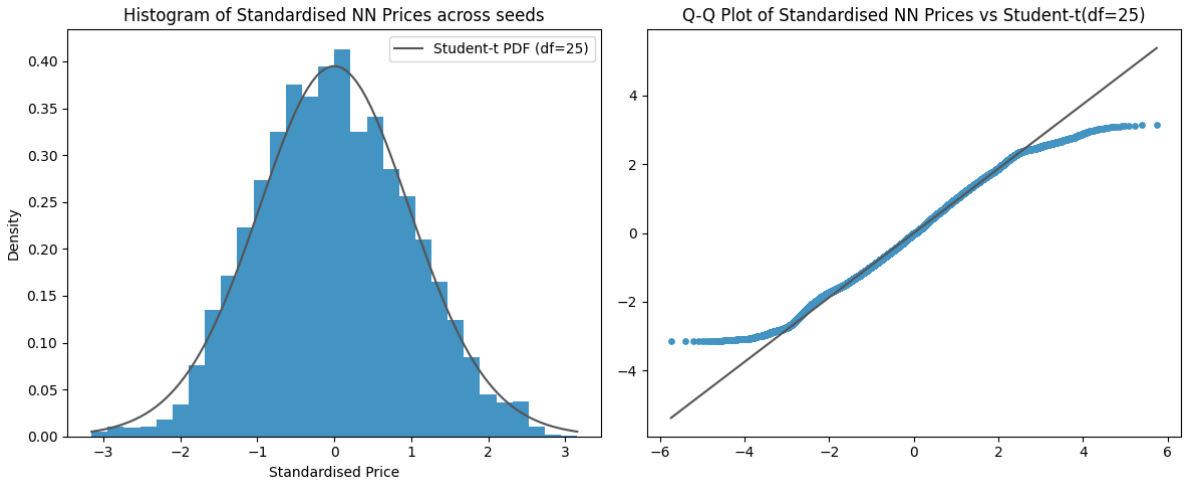

Figure 3.16 Distribution of standardised neural network price estimates across 25 runs

By looking at the histogram and the Q-Q plot, we can see that the distribution of the standardised price estimates is reasonably close to a t-distribution, with the exception of the tails. We

see that the observed distribution has lighter tails than the theoretical t-distribution. This is not ideal, but it is at least more statistically acceptable than heavier tails. This gives us grounds to proceed (with caution) and attempt to evaluate the free boundary using the Gaussian method. The right figure in Figure 3.15 shows the continuation probabilities across different (t, S) pairs.

We can see that the shape roughly corresponds to the free boundaries we have seen before, but this is a slightly noisier estimate. Both of these miss the true free boundary which lies between 0.7 and 0.8 at $t = 0$, which brings into question the assumptions we have made. We are essentially assuming that the price estimates of the PINN are unbiased and normally distributed around the true price, which is quite strong and not true in practice, as we have already observed the convergence of the at-the-money price across training epochs to a biased value in Figure 3.8, and the zones where the PINN price is systematically above or below the tree price in Figure 3.6.

Two-dimensional case

Using the same methodology as the one-dimensional case, we plot our estimated free boundaries in Figure 3.17. This is done on the S^1-S^2 cross section at $t = 0$, with the additional reference line of $S^1 \times S^2 = K$, which should be a continuation region as it is at the money and is worthless under immediate exercise.

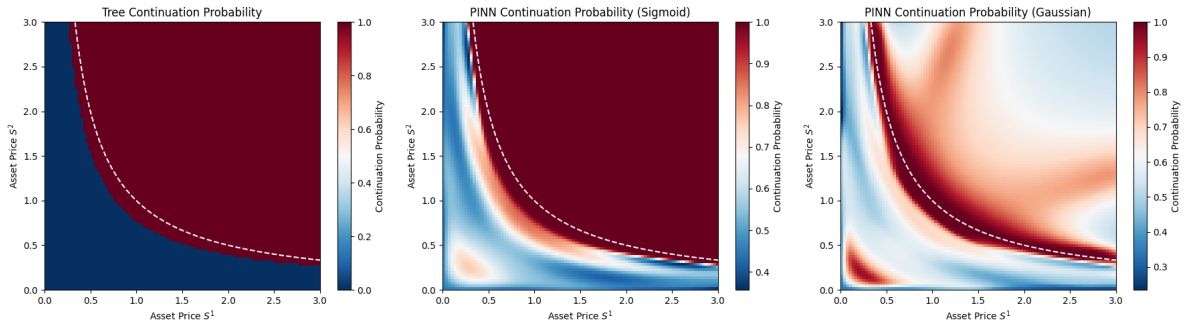

Figure 3.17 Estimated continuation probabilities for 2-dimensional case with $S^1 \times S^2 = K$ reference line

We can see that both methods of estimating the continuation probabilities yield similar shapes to the binomial tree benchmark, with the Gaussian method being noisier than the sigmoid method. We can further examine the continuation probabilities over time along the $S^1 = S^2$ cross section, varying across time. This is shown in Figure 3.18, where we can see that the PINN has successfully captured the structure of the free boundary across time where $S^1 = S^2$.

3.6 Minimum of two assets

To demonstrate the flexibility of our PINN framework, we will now shift our focus onto different payoffs on multiple assets as an extension. [30] explores a variety of different payoffs for American options on multiple assets, but the payoff with a particularly interesting free boundary is the call on the maximum of two assets. We will deal with the analogous case for the put on the minimum of two assets.

To price this, we can simply change the payoff function in our loss function, and update the boundary conditions accordingly to the new payoff function, but the variational inequality and the rest of the framework remain unchanged. For the American put on the minimum of two

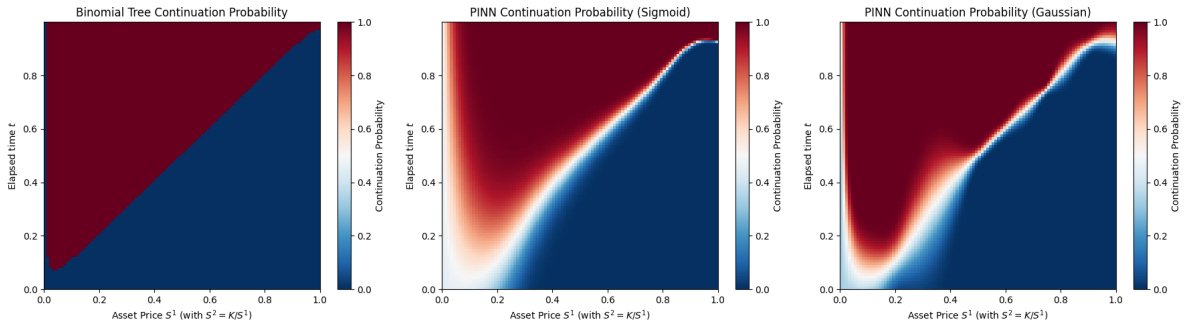

Figure 3.18 Estimated continuation probabilities for 2-dimensional case on $S^1 = S^2$ cross section across time

assets, the payoff function is given by $g\left(S^1, S^2\right) = \left(K - \min(S^1, S^2)\right)^+$, and thus we must have the terminal boundary condition

$$v\left(T, S^1, S^2\right) = \left(K - \min(S^1, S^2)\right)^+ \tag{3.84}$$

When one of the assets is very high (i.e., $S^1 \to \infty$ or $S^2 \to \infty$), the minimum of the two assets is just the other asset, and thus we have the reduction

$$v\left(t, \infty, S^2\right) = v_{1\mathrm{D}}\left(t, S^2\right) \qquad v\left(t, S^1, \infty\right) = v_{1\mathrm{D}}\left(t, S^1\right) \tag{3.85}$$

where $v_{1\mathrm{D}}$ is the price of the one-dimensional American put option on a single asset, which we can compute with the different methods we have already explored. Finally, when one of the assets is worthless, the minimum of the two assets is just the worthless asset, and hence the value will be K as we should exercise immediately. Thus, the boundary condition would be

$$v\left(t, S^1, S^2\right) = K \qquad \text{for } S^1 = 0 \text{ or } S^2 = 0 \tag{3.86}$$

Together, these boundary conditions (alongside the variational inequality which remains the same with the operator $\mathcal{L}$ as in (3.59)) form the PINN framework which we can train.

In Figure 3.19, we can see that our PINN yields results that are in line with our expectations. We expect to see some symmetry along the $S^1 = S^2$ line as the payoff is symmetric in the two assets. We introduce some asymmetry by training the network where S^2 has a higher constant volatility than S^1.

We can also see that the term structure is somewhat captured by the contour lines of the right figure of Figure 3.19, where the price decays as we approach maturity. We train the model with a high volatility of $(\sigma_1, \sigma_2) = (0.4, 0.5)$ to make the theta decay more significant. This also gives us a clearer evolution of the free boundary, which we see in Figure 3.20, where we can see that not only is the continuation region a mirror image of the theoretical result described in [30] as we would expect, but the PINN also captures the shrinking nature of the continuation region as we approach maturity, consistent with the intuition that the option loses time value and the region would close entirely, leaving only the box $\{(S^1, S^2) : S^1 \geq K, S^2 \geq K\}$ as the continuation region at maturity.

Overall, this demonstrates the capacity of our PINN approach to pricing options and we can see that the framework is flexible enough to be applied to different payoffs with minimal adjustments, yielding convincing results that are consistent with theoretical results. For the next step, we will move on to a different model, the Heston model, which injects randomness into the volatility, which is a more realistic assumption.

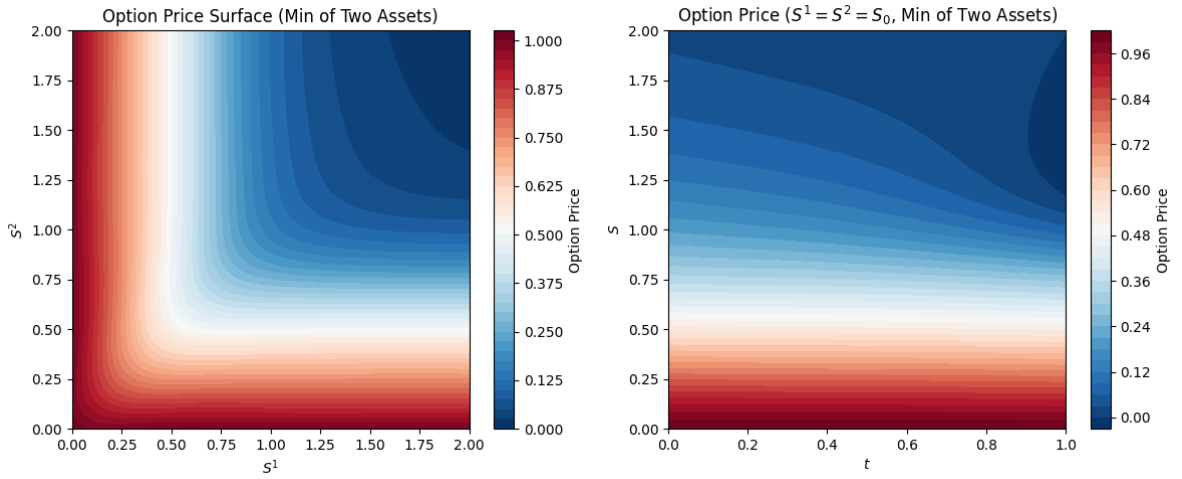

Figure 3.19 Heatmap for the put on the minimum of two assets across the S^1-S^2 cross section at $t = 0$, and the S^1-t cross section with $S^1 = S^2$

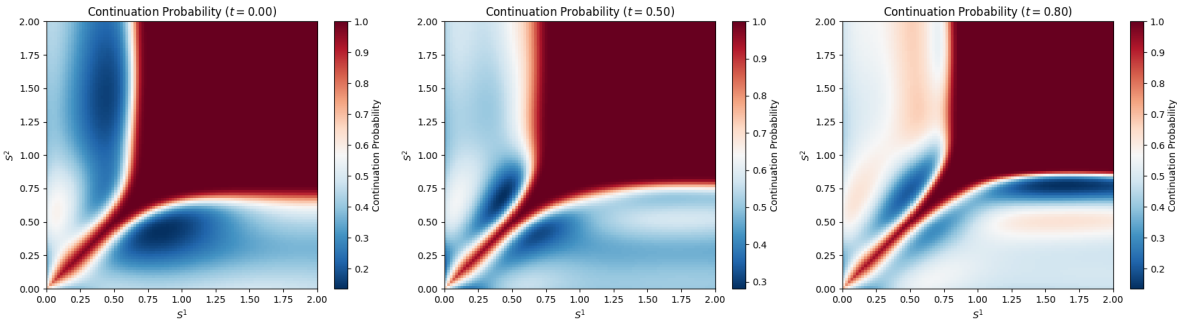

Figure 3.20 Continuation probabilities for the put on the minimum of two assets across the S^1-S^2 cross section with varying t

Chapter 4

The Heston Model

The classic Black-Scholes model makes the strong assumption that the stock returns are normally distributed with known mean and variance. However, the Black-Scholes formula does not depend on the mean of the log returns μ (since we perform a change of measure, and it depends instead on the risk-free rate r). We therefore have two directions in which we can take this project: we can model the risk-free rate as a stochastic process, or we can allow the volatility to vary according to a stochastic process. We will focus on the latter. Heston [2] proposes a model involving a stochastic variance process in which a closed-form solution for European call options exists. In this model, the dynamics under the pricing measure of the asset price S and the variance (squared volatility) process V are governed by the stochastic differential equation system under the risk-neutral measure $\mathbb{Q}$:

$$\mathrm{d}S_t = rS_t\mathrm{d}t + S_t\sqrt{V_t}\mathrm{d}W_t^1 \tag{4.1}$$

$$\mathrm{d}V_t = \kappa(\theta - V_t)\mathrm{d}t + \sigma\sqrt{V_t}\mathrm{d}W_t^2 \tag{4.2}$$

where we have initial conditions $S_0 > 0$ and $V_0 \geq 0$, and the two Brownian motions are correlated with

$$\mathrm{d}\langle W^1, W^2\rangle_t = \rho\,\mathrm{d}t \qquad \rho \in (-1, 1) \tag{4.3}$$

The dynamics of the variance V follow a Cox-Ingersoll-Ross (CIR) process with mean reversion rate $\kappa > 0$, long run state $\theta > 0$ and volatility of the volatility $\sigma > 0$. We can enforce the Feller condition $2\kappa\theta \geq \sigma^2$ so that the volatility process stays positive.

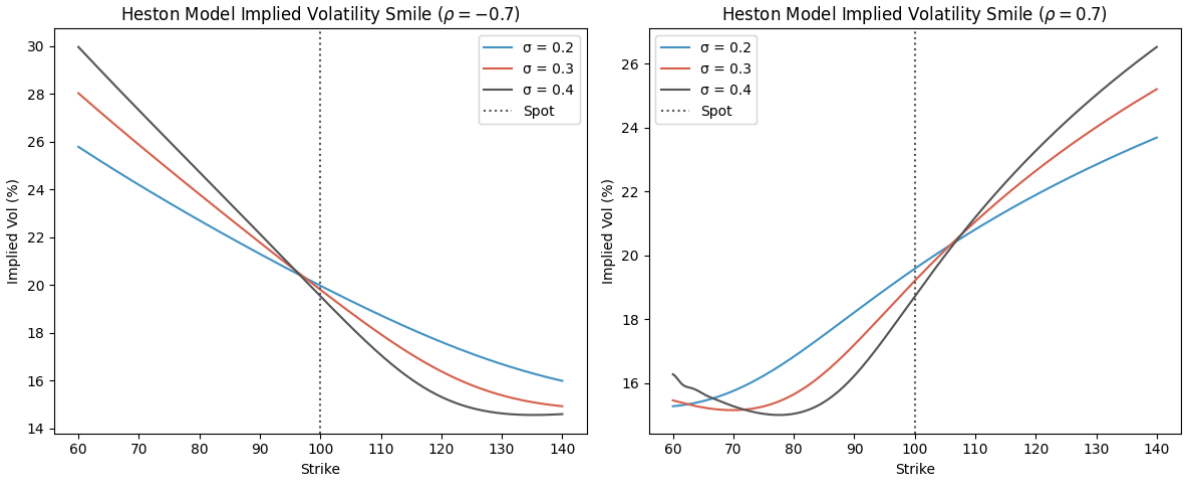

Figure 4.1 Volatility smile generated by the Heston model

The Heston model takes into account the stochastic nature of volatility while enforcing its mean-reverting nature, which is observed empirically in the market. It supports non-constant volatility smiles, as we can observe in Figure 4.1. The curvature is determined by the volatility of volatility σ, and the skew is determined by the correlation ρ.

4.1 Closed-form Solution

There exists a closed-form solution derived by Heston [2] to the price of the European call option (and indeed the put option, by put-call parity). This will serve as a benchmark, which would give us confidence in the accuracy of our other techniques for European options, and by extension, for American options too.

The closed-form solution is derived by using an Ansatz inspired by the Black-Scholes formula, where the unknown functions in the Ansatz can be determined by substituting our Ansatz into the corresponding PDE and boundary conditions. We will include a sketch of the derivation here which follows those found in [2, 31], but with some further justification for a better overall understanding.

Let C denote the price of the European call option. We will derive this PDE when we derive the variational equation in Section 4.3.1, where we show that the price of the European call option with strike K and maturity T must satisfy

$$\frac{\partial C}{\partial t} + rS\frac{\partial C}{\partial S} + \kappa(\theta - V)\frac{\partial C}{\partial V} + \frac{1}{2}VS^2\frac{\partial^2 C}{\partial S^2} + \rho\sigma VS\frac{\partial^2 C}{\partial S \partial V} + \frac{1}{2}\sigma^2 V\frac{\partial^2 C}{\partial V^2} - rC = 0 \tag{4.4}$$

with the terminal condition $C(T, S, V) = (S - K)^+$.

We will mimic the form of the Black-Scholes formula to form the Ansatz for the solution

$$C(S, V, t) = SP_1(x, V, t) - Ke^{-r(T-t)}P_2(x, V, t), \qquad x := \ln S \tag{4.5}$$

with terminal conditions $P_j(x, V, T) = \mathbf{1}_{\{x \geq \ln K\}}$ so that our Ansatz reduces to $(S - K)^+$ at maturity. We can substitute the Ansatz into the PDE and equate the coefficients of S and $Ke^{-r(T-t)}$ to obtain two PDEs for P_1 and P_2:

$$\frac{\partial P_j}{\partial t} + (r + u_j V)\frac{\partial P_j}{\partial x} + (a - b_j V)\frac{\partial P_j}{\partial V} + \frac{1}{2}V\frac{\partial^2 P_j}{\partial x^2} + \rho\sigma V\frac{\partial^2 P_j}{\partial x \partial V} + \frac{1}{2}\sigma^2 V\frac{\partial^2 P_j}{\partial V^2} = 0 \tag{4.6}$$

where $u_1 = \frac{1}{2}$, $u_2 = -\frac{1}{2}$, $a = \kappa\theta$, $b_1 = \kappa - \rho\sigma$ and $b_2 = \kappa$ [2]. With this in mind, let's define the characteristic functions

$$\phi_j(x, V, t; \omega) = \mathbb{E}^{\mathbb{Q}_j}\left[e^{i\omega x_T} \mid \mathcal{F}_t\right] \tag{4.7}$$

which admits the Ansatz

$$\phi_j(x, V, t; \omega) = \exp\left(C_j(\tau, \omega) + D_j(\tau, \omega)V + i\omega x\right), \qquad \tau := T - t \tag{4.8}$$

since the coefficients in the PDE for P_j are independent of x (justifying the $e^{i\omega x}$ term) and affine in V (justifying the $e^{D_j V}$ term)[32].

We can insert this Ansatz into the PDE for P_j to get a system of ODEs for C_j and D_j:

$$\frac{\partial D_j}{\partial \tau} = \frac{1}{2}\sigma^2 D_j^2 + (\rho\sigma i\omega - b_j)D_j + u_j i\omega - \frac{1}{2}\omega^2, \qquad \frac{\partial C_j}{\partial \tau} = ri\omega + aD_j \tag{4.9}$$

with the boundary conditions $C_j(0, \omega) = D_j(0, \omega) = 0$. There exists a closed-form solution to this system of ODEs, where we first define

$$d_j := \sqrt{(\rho\sigma i\omega - b_j)^2 - \sigma^2(2u_j i\omega - \omega^2)} \tag{4.10}$$

$$g_j := \frac{b_j - \rho\sigma i\omega - d_j}{b_j - \rho\sigma i\omega + d_j} \tag{4.11}$$

where we use the opposite sign convention compared to Heston's original formulation for better numerical stability (more precisely to avoid regions of numerical discontinuity in the complex plane) as suggested in [33]. This yields the solutions

$$D_j(\tau,\omega) = \frac{b_j - \rho\sigma i\omega - d_j}{\sigma^2} \cdot \frac{1 - e^{-d_j\tau}}{1 - g_j e^{-d_j\tau}} \tag{4.12}$$

$$C_j(\tau,\omega) = ri\omega\tau + \frac{a}{\sigma^2}\left[(b_j - \rho\sigma i\omega - d_j)\tau - 2\ln\frac{1 - g_j e^{-d_j\tau}}{1 - g_j}\right] \tag{4.13}$$

By the Gil-Pelaez inversion theorem [34], we can retrieve P_j from its characteristic function ϕ_j:

$$P_j = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Re\left[\frac{e^{-i\omega\ln K}\phi_j(x,V,t;\omega)}{i\omega}\right]\mathrm{d}\omega \tag{4.14}$$

which we can substitute back into our original Ansatz for the call option (4.5) to get the closed-form solution for the call option price. If we would like to be pedantic, we could argue this is not strictly closed-form as we require numerical integration to compute the second term of (4.14), but this remains a semi-analytical and tractable solution nonetheless, that we could use as a benchmark for our other methods.

4.2 Tree Solution

Since the closed-form solution is limited to European options, we would like to find ways to approximate solutions to the pricing problem so that we can eventually extend the case to American options, which have no closed-form solution. The first method we will explore is the tree method, where it is easy to modify to the American case by adding an additional comparison to the intrinsic value at each node. We will use the closed-form formula as a benchmark for the tree method for European options, to give us confidence in the solution the tree model provides for American options as it is a simple modification in the algorithm, which we can use as a benchmark for the neural networks we will explore later.

The idea will be similar to the binomial tree we explored earlier in the paper for the Black-Scholes model, but we now have two correlated stochastic processes in the Heston model: the asset price process and the variance process. As this tree is not recombinant, generating paths for these processes and computing the option prices by backward induction naively would lead to an exponential time complexity, which is not computationally feasible. Therefore, we will use a grid interpolation approach as in [4] so that the number of nodes grows quadratically in the number of time steps instead.

For the algorithm, we will work under the variance process V and log stock price process Z (which is a simple transformation from S using Itô's formula):

$$\mathrm{d}V_t = \kappa(\theta - V_t)\mathrm{d}t + \sigma\sqrt{V_t}\mathrm{d}W_t^1 \tag{4.15}$$

$$\mathrm{d}Z_t = (r - \tfrac{1}{2}V_t)\mathrm{d}t + \sqrt{V_t}\mathrm{d}W_t^2 \tag{4.16}$$

where $\mathrm{d}\langle W^1, W^2\rangle_t = \rho\,\mathrm{d}t$ under the risk-neutral measure $\mathbb{Q}$ and $\kappa, \sigma, \theta, r > 0$ as before. Of course, the value C of the European option with maturity T and payoff function in terms of stock and volatility values $g : \mathbb{R}^+ \times \mathbb{R}^+ \to \mathbb{R}$ will be

$$C(t, V_t, Z_t) = e^{-r(T-t)}\mathbb{E}^{\mathbb{Q}}[g(e^{Z_T}, \sqrt{V_T}) \mid \mathcal{F}_t] \tag{4.17}$$

4.2.1 Euler scheme

To construct the tree, we need to discretise the process. If we have n steps, then $\Delta t = \frac{T}{n}$. Using the standard Euler scheme, we can have

$$V_{k+1} = V_k + \kappa(\theta - V_k^+)\Delta t + Y_{k+1}^1 \sigma \sqrt{V_k^+ \Delta t} \tag{4.18}$$

$$Z_{k+1} = Z_k + (r - \tfrac{1}{2}V_k^+)\Delta t + Y_{k+1}^2 \sqrt{V_k^+ \Delta t} \tag{4.19}$$

where the increment variables (Y_k^1, Y_k^2) are iid pairs across time steps k with the joint distribution given by

$$\mathbb{Q}^n(Y_k^1 = i_1, Y_k^2 = i_2) = \frac{1}{4}(1 + i_1 i_2 \rho) \qquad i_1, i_2 \in \{-1, 1\} \tag{4.20}$$

We use a different measure $\mathbb{Q}^n$ (with n indicating the number of time steps) as it is not the same as the risk-neutral measure in the Heston model. To ensure positivity of the process, we have taken the positive part of the variance process $V_k^+ := \max(0, V_k)$. Note here, V_k shouldn't be negative anyway since we will enforce the Feller condition in practice, but with this in place we have full generality.

4.2.2 Interpolation optimisation

The problem with the naïve Euler scheme is that the tree is not recombining, and hence the number of nodes scales exponentially with increasing time steps, which is not feasible computationally. We will hence, for each time step k, construct a grid of option prices at time k and the backward induction will be a bilinear interpolation of the four nearest points.

To set this up, we need the boundaries of the grid at each time step k, which is given by the minimum and maximum values of V_k and Z_k in the support. In other words, we have

$$z_k^{\max} = \max\{z : \mathbb{Q}^n(Z_k = z) > 0\} \tag{4.21}$$

$$z_k^{\min} = \min\{z : \mathbb{Q}^n(Z_k = z) > 0\} \tag{4.22}$$

$$v_k^{\max} = \max\{v : \mathbb{Q}^n(V_k = v) > 0\} \tag{4.23}$$

$$v_k^{\min} = \min\{v : \mathbb{Q}^n(V_k = v) > 0\} \tag{4.24}$$

With these boundaries, we have the grid spacing given our mesh sizes $m_z, m_v \in \mathbb{N}^+$:

$$\Delta z_k = (z_k^{\max} - z_k^{\min})/m_z \tag{4.25}$$

$$\Delta v_k = (v_k^{\max} - v_k^{\min})/m_v \tag{4.26}$$

and the gridpoints:

$$\hat{\mathcal{S}}_k = \left\{ \left(v_k^{\min} + i\Delta v_k, z_k^{\min} + j\Delta z_k \right) \;\middle|\; i \in \{0, \ldots, m_v\}, j \in \{0, \ldots, m_z\} \right\} \tag{4.27}$$

We will now define the discretised process $(\tilde{V}, \tilde{Z})$ on the grid space $\hat{\mathcal{S}}_k$. This will involve the Euler scheme functions

$$\nu(y, v, z) = v + \kappa(\theta - v^+)\Delta t + y\sigma\sqrt{v^+ \Delta t} \tag{4.28}$$

$$\zeta(y, v, z) = z + (r - \tfrac{1}{2}v^+)\Delta t + y\sqrt{v^+ \Delta t} \tag{4.29}$$

where we notice ν doesn't depend on z, but we will put z in as an argument nonetheless for a uniform function signature.

Then using these functions, we can retrieve the next step. We can evolve our process to the next step with

$$(\tilde{V}_{k+1}, \tilde{Z}_{k+1}) = \Phi^{\hat{\mathcal{S}}_{k+1}}_{J^v_{k+1}, J^z_{k+1}} \left(\nu(Y^1_{k+1}, \tilde{V}_k, \tilde{Z}_k), \zeta(Y^2_{k+1}, \tilde{V}_k, \tilde{Z}_k)\right) \tag{4.30}$$

where $\Phi^{\mathcal{S}}_{j_1,j_2} : \mathbb{R}^2 \to \mathcal{S} \subset \mathbb{R}^2$ is a function mapping $(v, z) \in \mathbb{R}^2$ to one of the grid points in $\mathcal{S} := \mathcal{S}_v \times \mathcal{S}_z$. $j_1, j_2 \in \{0, 1\}$ are additional parameters that encode the snapping direction and determine the corner in which (v, z) is mapped, corresponding to J^v_{k+1}, J^z_{k+1} in (4.30). Visually, this is illustrated in Figure 4.2, where each point in the price-variance space can evolve to four different points in the next time step. For each of these four different points, we will take an interpolation of the four nearest grid points. Hence, we will consider sixteen points in total, some of which may be repeated.

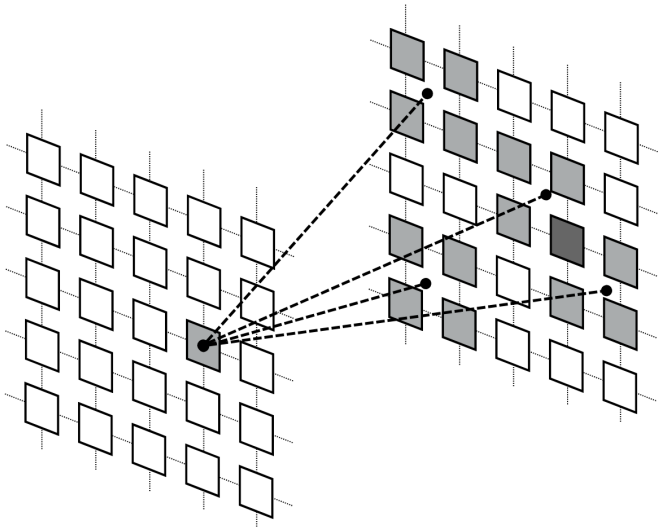

Figure 4.2 Interpolation illustration (adapted from [4])

To do this, we define the 1D snapping functions $\phi^{\mathcal{T}}_i : \mathbb{R} \to \mathcal{T}$ where we have the one-dimensional grid points $\mathcal{T} \subset \mathbb{R}$ and the snap-right indicator $i \in \{0, 1\}$. The function is defined mathematically as

$$\phi^{\mathcal{T}}_0(u) = \max\{u' \in \mathcal{T} : u' \leq u\} \tag{4.31}$$

$$\phi^{\mathcal{T}}_1(u) = \min\{u' \in \mathcal{T} : u' \geq u\} \tag{4.32}$$

Our 2D snapping function $\Phi^{\mathcal{S}}_{j_1,j_2} : \mathbb{R}^2 \to \mathcal{S}$ where $\mathcal{S} = \mathcal{S}_v \times \mathcal{S}_z \subset \mathbb{R}^2$ is thus

$$(v, z) \mapsto \left(\phi^{\mathcal{S}_v}_{j_1}(v), \phi^{\mathcal{S}_z}_{j_2}(z)\right) \tag{4.33}$$

The joint probabilities [4] for Y_{k+1} are given by, for $i_1, i_2 \in \{-1, 1\}, j_1, j_2 \in \{0, 1\}$:

$$q_{i_1,i_2,j_1,j_2}(\tilde{V}_k, \tilde{Z}_k) := \tilde{\mathbb{Q}}(Y^1_{k+1} = i_1, Y^2_{k+1} = i_2, J^v_{k+1} = j_1, J^z_{k+1} = j_2 \mid (\tilde{V}_k, \tilde{Z}_k)) \tag{4.34}$$

$$= \frac{1}{4}(1 + i_1 i_2 \rho) \times c^{\hat{\mathcal{S}}_{k+1}}_{j_1,j_2} \left(\nu(i_1, \tilde{V}_k, \tilde{Z}_k), \zeta(i_2, \tilde{V}_k, \tilde{Z}_k)\right) \tag{4.35}$$

where we have defined the measure $\tilde{\mathbb{Q}}$ as an extension of $\mathbb{Q}^n$ to include the random snapping indicators J^v_{k+1}, J^z_{k+1} as well, with the snapping probabilities given by the interpolation quantity [35] on a general grid $\mathcal{S} = \mathcal{S}_v \times \mathcal{S}_z \subset \mathbb{R}^2$:

$$c^{\mathcal{S}}_{j_1,j_2}(v, z) = \tilde{v}^{j_1}(1 - \tilde{v})^{1-j_1} \cdot \tilde{z}^{j_2}(1 - \tilde{z})^{1-j_2} \tag{4.36}$$

where we have the difference terms

$$\tilde{v} = \frac{v - \phi_0^{\mathcal{S}_v}(v)}{\phi_1^{\mathcal{S}_v}(v) - \phi_0^{\mathcal{S}_v}(v)} \tag{4.37}$$

$$\tilde{z} = \frac{z - \phi_0^{\mathcal{S}_z}(z)}{\phi_1^{\mathcal{S}_z}(z) - \phi_0^{\mathcal{S}_z}(z)} \tag{4.38}$$

By using the fact our grid is evenly spaced at all time steps k, this simplifies nicely to

$$\tilde{v} = \frac{v - \phi_0^{\mathcal{S}_v}(v)}{\Delta v_k}, \qquad \tilde{z} = \frac{z - \phi_0^{\mathcal{S}_z}(z)}{\Delta z_k} \tag{4.39}$$

4.2.3 Backward induction

We can now work backwards on the tree, having constructed the grids. Our base case is the final grid, where the price is just the payoff as we are at maturity. For a general time t_k at step k and node $(v, z) \in \hat{\mathcal{S}}_k$, we will generate the four possible steps determined by i_1, i_2 each taking values in $\{-1, 1\}$, and evolve our node with

$$(v, z) \mapsto \big(\nu(i_1, v, z), \zeta(i_2, v, z)\big) =: (v'_{i_1}, z'_{i_2}) \tag{4.40}$$

which will be in between four points on the grid $\hat{\mathcal{S}}_{k+1}$. These can be found by performing the Φ function on our evolved node: for $j_1, j_2 \in \{0, 1\}$ define the multi-index $\alpha = (i_1, i_2, j_1, j_2)$ and we have

$$(v_\alpha, z_\alpha) = \Phi_{j_1,j_2}^{\hat{\mathcal{S}}_{k+1}}(v'_{i_1}, z'_{i_2}) \tag{4.41}$$

Each of these will have a known option price. Therefore, we can just compute the discounted expectation of these sixteen nodes (two choices for each element of α) for the option price at the original node:

$$C(t_k, v, z) = e^{-r\Delta t} \sum_{\alpha \in \mathcal{A}} q_\alpha(v, z) \times C(t_{k+1}, v_\alpha, z_\alpha) \tag{4.42}$$

where $\mathcal{A} = \{-1, 1\}^2 \times \{0, 1\}^2$.

4.2.4 Implementation techniques

If we implemented this method naively, it would be very difficult for us to use the binomial tree as a benchmark because it would need to construct a new tree from scratch for every single value of S_0, V_0 and T. Thus any heatmaps or plots we create would require hundreds of trees which is not feasible, considering each tree requires many time steps and large mesh sizes to be accurate.

Thus in addition to vectorising our code for each tree, we will instead consider an initial range $[S_0^{\text{min}}, S_0^{\text{max}}]$ and $[V_0^{\text{min}}, V_0^{\text{max}}]$ which we will use to construct our grid bounds. More specifically, z_k^{max} and v_k^{max} will be the up-most path starting with $(S_0^{\text{max}}, V_0^{\text{max}})$ instead of simply some individual value (S_0, V_0), and similarly for the down paths. In this way, when we use the binomial tree as a benchmark, we can use any initial value of (S_0, V_0) as long as it's in the original range and compute the price with an interpolation as described above. With this method, we can evaluate across different initial values while constructing just one tree. We sacrifice a little accuracy in doing so because the grids would be more widely spaced, which we can combat by increasing the mesh sizes. Thus, we effectively construct one more complex tree instead of hundreds of simpler ones, which is much more computationally efficient.

As for selecting our mesh sizes, we will use the result that if we have

$$\lim_{n\to\infty} \Delta t = 0, \qquad \lim_{n\to\infty} \max_{k=1,\dots,n-1} \frac{\Delta v_k^n}{\Delta t_k} = 0, \qquad \lim_{n\to\infty} \max_{k=1,\dots,n-1} \frac{\Delta z_k^n}{\Delta t_k} = 0 \tag{4.43}$$

then the process represented in our Heston tree converges weakly to the original process (V, Z) [4]. Note here we denote Δv_k from before as Δv_k^n to emphasise we have split our maturity T into n steps, and similarly for Δz_k^n.

Bearing this in mind, a resolution of $n = 100, m_v = 300, m_z = 600$ allows our tree to faithfully reproduce the closed-form solution, which is presented in the numerical results in Section 4.5.1.

4.3 PINN Solution

4.3.1 Variational equation

We would now like to derive the variational equation for options on assets under the Heston model. Note that for a d-dimensional general process $X_t \in \mathbb{R}^d$ solving the Itô SDE

$$\mathrm{d}X_t = b(X_t)\mathrm{d}t + \sigma(X_t)\mathrm{d}W_t \tag{4.44}$$

where we have the drift $b : \mathbb{R}^d \to \mathbb{R}^d$, the volatility $\sigma : \mathbb{R}^d \to \mathbb{R}^{d\times m}$ and the m-dimensional Brownian motion W_t. On second-order smooth functions $f \in \mathcal{C}^2(\mathbb{R}^d)$, the infinitesimal generator $\mathcal{L}$ is defined as [36]

$$\mathcal{L}[f](x) := \lim_{t\to 0} \frac{\mathbb{E}[f(X_t) \mid X_0 = x] - f(x)}{t} \tag{4.45}$$

which exists and can be derived with Itô's formula (or Taylor's expansion) to be [12]

$$\mathcal{L}[f](x) = \sum_{i=1}^{d} b_i \frac{\partial f}{\partial x_i} + \frac{1}{2}\sum_{i=1}^{d}\sum_{j=1}^{d} \Sigma_{ij} \frac{\partial^2 f}{\partial x_i \partial x_j} \tag{4.46}$$

where b_i is the i-th component of the drift vector $\boldsymbol{b} := b(x)$ and Σ_{ij} is the (i,j)-th component of the covariance matrix $\Sigma(x) = \boldsymbol{\sigma}\boldsymbol{\sigma}^\top$, with $\boldsymbol{\sigma} := \sigma(x)$.

Let's apply this to our Heston model. The drift vector is

$$b(S, V) = \begin{pmatrix} rS \\ \kappa(\theta - V) \end{pmatrix} \tag{4.47}$$

To compute $\sigma\sigma^\top$, we need to represent our correlated Brownian motions in terms of independent ones. Suppose we have independent Brownian motions B^1 and B^2, we can define

$$W^1 = B^1 \tag{4.48}$$

$$W^2 = \rho B^1 + \sqrt{1-\rho^2}B^2 \tag{4.49}$$

which has the quadratic variation $\mathrm{d}\langle W^1, W^2\rangle_t = \rho \mathrm{d}t$ as required. Using these new Brownian motions, we can rewrite the SDE as

$$\mathrm{d}\begin{pmatrix} S_t \\ V_t \end{pmatrix} = \underbrace{\begin{pmatrix} rS_t \\ \kappa(\theta - V_t) \end{pmatrix}}_{b(S_t,V_t)} \mathrm{d}t + \underbrace{\begin{pmatrix} S_t\sqrt{V_t} & 0 \\ \rho\sigma\sqrt{V_t} & \sigma\sqrt{V_t}\sqrt{1-\rho^2} \end{pmatrix}}_{\sigma(S_t,V_t)} \begin{pmatrix} \mathrm{d}B_t^1 \\ \mathrm{d}B_t^2 \end{pmatrix} \tag{4.50}$$

which means that our volatility matrix is now

$$\boldsymbol{\sigma} = \sigma(S,V) = \begin{pmatrix} S\sqrt{V} & 0 \\ \rho\sigma\sqrt{V} & \sigma\sqrt{V}\sqrt{1-\rho^2} \end{pmatrix}. \tag{4.51}$$

which means

$$\Sigma = \boldsymbol{\sigma}\boldsymbol{\sigma}^\top = \begin{pmatrix} S^2V & \rho\sigma SV \\ \rho\sigma SV & \sigma^2 V \end{pmatrix} \tag{4.52}$$

Note here we have many forms of σ which may be confusing: σ without any arguments is the volatility of the variance process, $\sigma(S,V)$ is the volatility matrix of the SDE, and $\boldsymbol{\sigma}$ is the same as $\sigma(S,V)$ but with a bold symbol to distinguish it from the scalar σ.

We now have everything we need for the formula. For any function $f = f(S,V) \in \mathcal{C}^2(\mathbb{R}^2_+)$, we have

$$\mathcal{L}[f] = rS\frac{\partial f}{\partial S} + \kappa(\theta - V)\frac{\partial f}{\partial V} + \frac{1}{2}\left(S^2V\frac{\partial^2 f}{\partial S^2} + 2\rho\sigma SV\frac{\partial^2 f}{\partial S \partial V} + \sigma^2 V\frac{\partial^2 f}{\partial V^2}\right) \tag{4.53}$$

This is our infinitesimal generator $\mathcal{L}$ acting on smooth functions f. Suppose we have a European option with payoff $g : \mathbb{R}^+ \to \mathbb{R}^+$, then our price function $v(t,S,V)$, the value of the call option with expiry T at time t with $S_t = S$ and $V_t = V$ satisfies

$$\mathcal{G}[v] := rv - \frac{\partial v}{\partial t} - \mathcal{L}[v] = 0 \tag{4.54}$$

with terminal condition $v(T,S,V) = g(S)$. This matches the PDE we had stated in the closed-form solution in (4.4).

As a check, we can set $\kappa = 0$ and $\sigma = 0$ so that $\mathrm{d}V = 0$ and volatility becomes a constant, and our model collapses to what we have been working with before. Our generator becomes

$$\mathcal{L}[f] = rS\frac{\partial f}{\partial S} + \frac{1}{2}S^2V_0\frac{\partial^2 f}{\partial S^2} \tag{4.55}$$

which matches our expression (3.36) from the Black-Scholes section, since V_0 is the initial (and hence constant) variance.

We have derived the variational equation for the European option for simplicity, but it can be easily extended for the American case. With the same argument as (3.38), the value of the American option must satisfy the PDE

$$\min\big(\mathcal{G}[v], v(t,S,V) - g(S)\big) = 0 \tag{4.56}$$

4.3.2 PINN formulation

We can fit the variational equation (4.56) into the PINN framework established in Section 2.3 which will serve as the PDE loss (2.39), and all that is left is to set up the loss function and the relevant boundary conditions. It will be very similar to the Black-Scholes case, but as we have an additional dimension V for the variance, we will need additional boundary conditions to ensure regularity in the variance dimension as well. These boundary conditions will be inspired by those given for the European call option in [2], but we will adapt them in our study, as they exhibit different behaviour for American put options.

1. $t \to T$: the value at expiry is just the intrinsic value, or the payoff of the put option:

$$v(T,S,V) = \max(0, K - S) \tag{4.57}$$

which will form the terminal loss (2.40) for the PINN.

2. $S \to 0$: the option value is the strike when the stock is worthless, because we can exercise immediately (for European options we would require a discounting term):

$$v(t, 0, V) = K \tag{4.58}$$

3. $S \to \infty$: the value of a very far out-of-the-money option is 0, as it is very unlikely for it to expire in-the-money:

$$v(t, \infty, V) = 0 \tag{4.59}$$

4. $V \to 0$: volatility being zero is a slightly more complicated case, as this would only be true instantaneously due to the mean-reverting nature of the variance process, and the fact the variance process is guaranteed to be positive by the Feller condition. The original paper by Heston [2] proposes a collapsed first-order PDE, but this turns out to be a bit overkill for the one-dimensional case, as it doesn't yield better results. We will take this case to be that the asset price process becomes approximately deterministic, a simplification of small volatilities, and hence the value of the option can be taken to be the payoff

$$v(t, S, 0) = \max(0, K - S) \tag{4.60}$$

5. $V \to \infty$: when the volatility is very high, we will take it that it is almost sure that the asset price would be zero at some point before expiry, so the value of the option is the strike:

$$v(t, S, \infty) = K \tag{4.61}$$

which is the asymptotic condition.

In practice, as we clearly cannot represent infinity, we will settle for large constants for the asset price S_{inf} and variance V_{inf}. The minimum for both of these will be taken to be zero. Thus our domain is

$$\Omega = (0, T) \times (0, S_{\text{max}}) \times (0, V_{\text{max}}) \tag{4.62}$$

noting the difference between S_{max} and S_{inf}, where the former is the maximum asset price in our sampling domain, and the latter is the value we use for the boundary condition. This also applies to V_{max} and V_{inf}.

The boundary conditions (2)-(5) together will form the boundary loss (2.41) for the PINN. By choosing relevant weights for each of these loss components (we could additionally set different values for different individual boundary components), we can ensure that the neural network learns the correct behaviour at the boundaries, which is crucial for the accuracy of the solution.

4.4 Multi-asset PINN

With the neural network established, we are now ready to move on to pricing options on multiple assets. As with the Black-Scholes case, we will price the put on the product of multiple assets because we can find a one-dimensional reduction. Here, we will consider a simplified model.

Let us have n assets with price processes $S = (S^1, \dots, S^n)^\top$ which are correlated with each other with the $n \times n$ correlation matrix Σ. Suppose further we have a common variance process V which is correlated with each of the assets. Then this process is governed by the stochastic differential equations:

$$\mathrm{d}S = r\mathrm{diag}(S)\,\mathrm{d}t + \sqrt{V}\mathrm{diag}(S)\mathrm{diag}(\sigma)L\,\mathrm{d}W^1 \tag{4.63}$$

$$\mathrm{d}V = \kappa(\theta - V)\,\mathrm{d}t + \bar{\sigma}\sqrt{V}\,\mathrm{d}W^2 \tag{4.64}$$

where we have the vector of iid Brownian motions $W^1 = (W^{1,1}, \ldots, W^{1,n})^\top$, volatility vector $\sigma = (\sigma_1, \ldots, \sigma_n)^\top$ and the lower-triangular Cholesky decomposition of the correlation matrix L such that $\Sigma = LL^\top$ where its diagonal has non-negative entries. This is guaranteed to exist as Σ is positive semi-definite (more precisely, if $\Sigma \succ 0$ then L exists and is unique, and if $\Sigma \succeq 0$ then L exists but may not be unique).

For simplicity, in this model we will assume equicorrelation between the assets: that is $\Sigma_{ij} = \varrho$ for $i \neq j$ and $\Sigma_{ii} = 1$. Alternatively, we can write

$$\Sigma = (1 - \varrho) I_n + \rho \mathbf{1}\mathbf{1}^\top \tag{4.65}$$

where I_n is the n-dimensional identity matrix and $\mathbf{1}$ is the n-dimensional vector of ones. We will quickly derive a bound for ϱ to ensure $\Sigma \succeq 0$, so that it is a valid correlation matrix. As Σ has an eigenvector $\mathbf{1}$ with coresponding eigenvalue $\lambda_1 = 1 + (n-1)\varrho$ and $n-1$ eigenvectors orthogonal to $\mathbf{1}$ with corresponding eigenvalue $\lambda_2 = 1 - \varrho$, we have $\Sigma \succeq 0$ if and only if $\lambda_1, \lambda_2 \geq 0$, which gives us the bound $\varrho \in [-\frac{1}{n-1}, 1]$.

We will also assume that each independent driving factor is correlated with the variance process: $\mathrm{d}W^{1,j}\mathrm{d}W^2 = \rho_j \mathrm{d}t$, with $\rho = (\rho_1, \ldots, \rho_n)^\top$, the cross-correlation vector between assets and the variance process. Note that this means the asset S^i satisfies

$$\mathrm{d}S^i = S^i \left(r \, \mathrm{d}t + \sqrt{V}\sigma_i \sum_{j=1}^{n} L_{ij} \, \mathrm{d}W^{1,j} \right) \tag{4.66}$$

To guarantee positiveness of the variance process, we will enforce the Feller condition on the parameters

$$2\kappa\theta \geq \bar{\sigma}^2 \tag{4.67}$$

We make these assumptions to allow a one-dimensional reduction for the Heston model, which would provide us with a comparable benchmark for our results with the tree method. This would in turn give us confidence for our approximated solutions, should we wish to relax assumptions or to change the payoff.

4.4.1 One-dimensional reduction

Since we would like to find an equivalent formulation of the product of assets $X := \prod_{i=1}^n S^i$, it would be easier to work in the log domain. That is, find the governing SDE for the log process $Y = \log X = \sum_{i=1}^n \log S^i$ and then exponentiate with Itô's lemma to recover the desired result.

Let $Y^i = \log S^i$. Then by Itô's lemma, we have

$$\mathrm{d}Y^i = \frac{1}{S^i} \, \mathrm{d}S^i - \frac{1}{2{S^i}^2} \, \mathrm{d}\langle S^i \rangle_t \tag{4.68}$$

where ${S^i}^2$ is the square of the price of the i-th asset. Here, the quadratic variation term $\mathrm{d}\langle S^i \rangle_t$ is given by

$$\mathrm{d}\langle S^i \rangle_t = {S^i}^2 V \sigma_i^2 \sum_{j=1}^{n} \sum_{k=1}^{n} L_{ij} L_{ik} \, \mathrm{d}\langle W^{1,j}, W^{1,k} \rangle_t = {S^i}^2 V \sigma_i^2 \sum_{j=1}^{n} L_{ij}^2 \, \mathrm{d}t \tag{4.69}$$

and we have $\sum_{j=1}^n L_{ij}^2 = \Sigma_{ii} = 1$, thus giving us $\mathrm{d}\langle S^i \rangle_t = {S^i}^2 V \sigma_i^2 \, \mathrm{d}t$. Therefore, we have

$$\mathrm{d}Y^i = r \, \mathrm{d}t + \sqrt{V}\sigma_i \sum_{j=1}^{n} L_{ij} \, \mathrm{d}W^{1,j} - \frac{1}{2} V \sigma_i^2 \, \mathrm{d}t \tag{4.70}$$

We can collect terms and sum over i to get (recalling $Y = \sum_{i=1}^n Y^i$):

$$\mathrm{d}Y = \left(nr - \frac{1}{2} V \sum_{i=1}^n \sigma_i^2 \right) \mathrm{d}t + \sqrt{V} \sum_{j=1}^n \sum_{i=1}^n \sigma_i L_{ij} \, \mathrm{d}W^{1,j} \tag{4.71}$$

For ease of notation, let $A = \sum_{i=1}^n \sigma_i^2$ and $b_j = \sum_{i=1}^n \sigma_i L_{ij}$. Then we have

$$\mathrm{d}Y = \left(nr - \frac{1}{2} VA \right) \mathrm{d}t + \sqrt{V} \sum_{j=1}^n b_j \, \mathrm{d}W^{1,j} \tag{4.72}$$

Now Y has variance

$$\mathrm{d}\langle Y \rangle_t = V \sum_{j=1}^n b_j^2 \, \mathrm{d}t =: VB \, \mathrm{d}t \tag{4.73}$$

where we let $B := \sum_{j=1}^n b_j^2$ for convenience, we can rewrite $\mathrm{d}Y$ with a new Brownian motion:

$$\mathrm{d}Y = \left(nr - \frac{1}{2} VA \right) \mathrm{d}t + \sqrt{VB} \, \mathrm{d}W^X, \qquad \mathrm{d}W^X = \frac{\sum_{j=1}^n b_j \, \mathrm{d}W^{1,j}}{\sqrt{B}} \tag{4.74}$$

Note here that B can be expressed directly in terms of Σ, independently of the choice of the Cholesky factor L

$$B = \sum_{j=1}^n b_j^2 = \|\sigma^\top L\|^2 = \sigma^\top L L^\top \sigma = \sigma^\top \Sigma \sigma \tag{4.75}$$

which means we have

$$\mathrm{d}W^X \mathrm{d}W^2 = \frac{\sum_{j=1}^n b_j \rho_j}{\sqrt{B}} \, \mathrm{d}t \tag{4.76}$$

so we can derive an explicit formula for the scalar correlation between W^X and W^2:

$$\rho^X = \frac{\sum_{j=1}^n b_j \rho_j}{\sqrt{B}} = \frac{\sigma^\top L \rho}{\sqrt{\sigma^\top \Sigma \sigma}} \tag{4.77}$$

Using our equicorrelation structure $\Sigma_{ij} = \varrho$ for $i \neq j$ and $\Sigma_{ii} = 1$, we have the nice simplification for B:

$$B = \sigma^\top \Sigma \sigma = (1 - \varrho) A + \varrho \left(\sum_{i=1}^n \sigma_i \right)^2 \tag{4.78}$$

Now, we have everything we need to recover $X = e^Y$. Using Itô's lemma again, we have

$$\mathrm{d}X = X \, \mathrm{d}Y + \frac{1}{2} X \, \mathrm{d}\langle Y \rangle_t \tag{4.79}$$

$$= X \left(nr + \frac{1}{2} V(B - A) \right) \mathrm{d}t + X \sqrt{VB} \, \mathrm{d}W^X \tag{4.80}$$

This is close to the Heston model, with the diffusion $\tilde{V} = VB$ which gives us

$$\mathrm{d}\tilde{V} = B \, \mathrm{d}V \tag{4.81}$$

$$= \kappa(B\theta - \tilde{V}) \, \mathrm{d}t + \bar{\sigma} \sqrt{B} \sqrt{\tilde{V}} \, \mathrm{d}W^2 \tag{4.82}$$

which is a new CIR process $\tilde{V}$ with parameters $\tilde{\kappa} = \kappa$, $\tilde{\theta} = B\theta$ and $\tilde{\sigma} = \bar{\sigma}\sqrt{B}$ (as B is constant). The product of assets nearly gives a direct reduction to a one-dimensional Heston process, but

the drift of X contains V which is a stochastic process itself. Thus for our benchmark, we will set $\varrho = 0$ which means $B = A$ and the drift term becomes deterministic. The last thing to check is the Feller condition for $\tilde{V}$, which is satisfied as

$$2\tilde{\kappa}\tilde{\theta} \geq \tilde{\sigma}^2 \iff 2\kappa B\theta \geq \bar{\sigma}^2 B \iff 2\kappa\theta \geq \bar{\sigma}^2 \tag{4.83}$$

As such, we are able to recover a one-dimensional Heston model for the product of assets X with variance $\tilde{V}$ and drift nr.

4.4.2 Variational equation

Recall for a process $X_t \in \mathbb{R}^n$ solving $\mathrm{d}X_t = b(X_t)\mathrm{d}t + \sigma(X_t)\mathrm{d}W_t$, the infinitesimal generator $\mathcal{L}$ can be derived with Itô's formula [12] on $f(x)$ to be

$$\mathcal{L}[f](x) = \sum_{i=1}^{n} \big(b(x)\big)_i \frac{\partial f}{\partial x_i} + \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \Big(\sigma(x)\sigma(x)^\top\Big)_{ij} \frac{\partial^2 f}{\partial x_i \partial x_j} \tag{4.84}$$

In our case, we have a function $f = f(S^1, \ldots, S^n, V)$, and we can derive the infinitesimal generator much like the 1D case, but this time we need to sum over our different asset and variance processes. The drift vector is easy to see by looking at the SDEs, which is

$$b(S, V) = \begin{pmatrix} rS^1 \\ \vdots \\ rS^n \\ \kappa(\theta - V) \end{pmatrix} \tag{4.85}$$

We need to be more careful with the variance matrix, as our Brownian motions are correlated, and we need to write a variance matrix in terms of iid Brownian motions. Since $W^{1,1}, \ldots, W^{1,n}$ are already independent of each other, we only need to handle the correlation $\mathrm{d}\langle W^{1,j}, W^2\rangle_t = \rho_j \mathrm{d}t$. Consider

$$\mathrm{d}W^2 = \sum_{j=1}^{n} \rho_j \mathrm{d}W^{1,j} + \sqrt{1 - |\rho|^2}\,\mathrm{d}B^{n+1} \tag{4.86}$$

where we have $|\rho|^2 = \sum_{j=1}^{n} \rho_j^2$ and where B^{n+1} is a 1-dimensional Brownian motion, independent of all $W^{1,j}$. Clearly we need $|\rho|^2 \leq 1$ for this to be well-defined. Note this is quite a strong condition as it means the variance process cannot be too strongly correlated with all the assets at the same time, so care must be taken to ensure this holds before proceeding in practice.

We can easily verify that this satisfies the quadratic variation

$$\mathrm{d}\langle W^{1,j}, W^2\rangle_t = \rho_j \mathrm{d}t, \qquad \mathrm{d}\langle W^2, W^2\rangle_t = \mathrm{d}t \tag{4.87}$$

as needed. This gives us the needed $n + 1$ independent driving Brownian motions: we have $(B^1, \ldots, B^{n+1})$, with $B^i = W^{1,i}$ for $i = 1, \ldots, n$.

We now rewrite our SDEs in the multi-asset Heston model in terms of these independent Brownian motions. The assets S^i are not driven by B^{n+1} so we simply have

$$\mathrm{d}S^i = rS^i\mathrm{d}t + S^i\sqrt{V}\sigma_i \sum_{j=1}^{n} L_{ij}\mathrm{d}B^j + 0 \cdot \mathrm{d}B^{n+1} \tag{4.88}$$

and then we can substitute the decomposition of $\mathrm{d}W^2$ into the variance process to get

$$\mathrm{d}V = \kappa(\theta - V)\mathrm{d}t + \bar{\sigma}\sqrt{V}\sum_{j=1}^{n}\rho_j \mathrm{d}B^j + \bar{\sigma}\sqrt{V}\sqrt{1-|\rho|^2}\,\mathrm{d}B^{n+1} \tag{4.89}$$

We can now read off the $(n+1)\times(n+1)$ variance matrix

$$\sigma(S,V) = \begin{pmatrix} \sqrt{V}\,\mathrm{diag}(S)\,\mathrm{diag}(\sigma)L & 0_n \\ \bar{\sigma}\sqrt{V}\rho^\top & \bar{\sigma}\sqrt{V}\sqrt{1-|\rho|^2} \end{pmatrix} \tag{4.90}$$

which is written in block form: $\sqrt{V}\mathrm{diag}(S)\mathrm{diag}(\sigma)L$ is the $n\times n$ block for the assets, 0_n is the n-dimensional zero vector for the lack of contribution of the variance Brownian motion in the assets (due to independence), $\bar{\sigma}\sqrt{V}\rho^\top$ is the $1\times n$ block of the contribution of the asset Brownian motions to the variance process, and $\bar{\sigma}\sqrt{V}\sqrt{1-|\rho|^2}$ is the 1×1 block for the contribution of the variance Brownian motion to the variance process. We can then compute the covariance matrix $\tilde{\Sigma}$ (to differentiate it from Σ for the asset correlations) as

$$\tilde{\Sigma} = \sigma(S,V)\sigma(S,V)^\top = \begin{pmatrix} V\mathrm{diag}(S)\mathrm{diag}(\sigma)\Sigma\mathrm{diag}(\sigma)\mathrm{diag}(S) & V\bar{\sigma}\mathrm{diag}(S)\mathrm{diag}(\sigma)L\rho \\ V\bar{\sigma}(L\rho)^\top\mathrm{diag}(\sigma)\mathrm{diag}(S) & \bar{\sigma}^2 V \end{pmatrix} \tag{4.91}$$

Therefore, by plugging it into the formula (4.84), our generator can be expressed to be

$$\begin{aligned}\mathcal{L}[f] = &\sum_{i=1}^{n} rS^i\frac{\partial f}{\partial S^i} + \kappa(\theta - V)\frac{\partial f}{\partial V} + \frac{1}{2}V\sum_{i=1}^{n}\sum_{j=1}^{n}\sigma_i\sigma_j\Sigma_{ij}S^iS^j\frac{\partial^2 f}{\partial S^i\partial S^j} \\ &+ \frac{1}{2}\bar{\sigma}^2 V\frac{\partial^2 f}{\partial V^2} + \sum_{i=1}^{n}\bar{\sigma}\sigma_i(L\rho)_i V S^i\frac{\partial^2 f}{\partial S^i\partial V}\end{aligned} \tag{4.92}$$

For any payoff $g(\cdot)$, we have, as before, the variational equation for a European option being

$$\mathcal{G}[f] := rf - \frac{\partial f}{\partial t} - \mathcal{L}[f] = 0 \tag{4.93}$$

and for an American option, we can introduce the early stopping criterion and the variational equation becomes

$$\min(\mathcal{G}[f], f - g) = 0 \tag{4.94}$$

4.4.3 PINN formulation

This will be very similar to the one-dimensional case, but we will make a few modifications to account for the additional dimensions. The PDE loss will be the same as before, but the boundary conditions will include:

1. $t \to T$: the value at expiry is just the intrinsic value which is the payoff:

$$u(T, S^1, \ldots, S^n, V) = \max\left(0, K - \prod_{k=1}^{n} S^k\right) \tag{4.95}$$

   which serves as our terminal loss (2.40) for the PINN.

2. $S \to 0$: if one of the assets reaches zero, then the product of assets is consequently zero and we should exercise immediately. Thus the value is the strike. To implement this, let $\tilde{S}$ be S with one of the assets modified manually to 0. Then we have

$$v(t, \tilde{S}, V) = K \tag{4.96}$$

3. $S \to \infty$: if one of the assets has a very high price, then the product of assets would also be very high meaning the option would expire worthless out-of-the-money. To implement this, let $\bar{S}$ be S with one of the assets modified manually to S_{inf}, a large constant. Then the boundary condition is

$$v(t, \bar{S}, V) = 0 \tag{4.97}$$

4. $V \to 0$: the simplification in the one-dimensional case doesn't work well in practice for the multi-asset case, so we will consider the degenerate PDE as proposed in the original paper by Heston [2], where we have the complementary condition

$$\min\left(rv - \frac{\partial v}{\partial t} - r\sum_{i=1}^{n} S^i \frac{\partial v}{\partial S^i} - \kappa\theta\frac{\partial v}{\partial V}, v - g\right) = 0 \tag{4.98}$$

where we evaluate v at $(t, S, 0)$.

5. $V \to \infty$: with infinite volatility, we could enforce the asymptotic condition as in the one-dimensional case, where

$$v(t, S, V_{\text{inf}}) = K \tag{4.99}$$

for some large constant V_{inf}. However, in practice, the PINN struggles to learn this boundary condition, and hence we will use a softer version of this constraint, taking into account the first order:

$$\frac{\partial v}{\partial V}(t, S, V_{\text{max}}) = 0 \tag{4.100}$$

using the fact that the value of the option flattens as volatility increases, since our practical implementation of V_{max} is not large enough to be in the asymptotic region, and using V_{inf} creates a large gap between the practical domain and the asymptotic region, decreasing the accuracy of the solution in the practical domain.

4.5 Numerical Results

4.5.1 Interpolating tree method

As mentioned before, we choose the resolution $n = 100, m_v = 300, m_z = 600$ which allows a faithful approximation of the closed-form solution while keeping runtime low. Figure 4.3 shows the results after using the parameters $K = 1, T = 1, r = 0.05, \kappa = 2, \theta = 0.04, \sigma = 0.3, \rho = -0.7$. We produce these plots by varying across different values of S_0 (for delta), V_0 (for vega) and time elapsed (for term structure), and all else kept constant.

Figure 4.3 Call option prices by the Heston Tree

We can see that the Heston tree does indeed approximate the closed-form solution well, with a small error. This gives us confidence in our implementation, and with an easy modification we can compute the American option prices which we will use as a benchmark.

To compute the American option price with the tree model, all we need to do is to introduce the stopping condition at every node: we evaluate the intrinsic value of the option at each node and set the option value to be the maximum of the intrinsic value and the discounted continuation value.

In Figure 4.4 we can see that the European option price indeed bounds the American prices from below, as we would expect. We have confidence that our American prices are accurate since our binomial tree faithfully approximates the closed-form solution for European options. We will now use this as a benchmark for neural network approximations.

Figure 4.4 American vs European option prices for the put

4.5.2 One-dimensional PINN solution

We are now ready to train the neural network to price our option. We choose to use a learning rate of 1×10^{-3} with the Adam optimiser and a scheduler with step size 2000 and gamma 0.7 and the Sigmoid activation function. By using parameters $K = 1, T = 1, r = 0.1, \kappa = 2, \theta = 0.04, \sigma = 0.3, \rho = -0.7$, we obtain the plots as shown in Figure 4.5 for the American put.

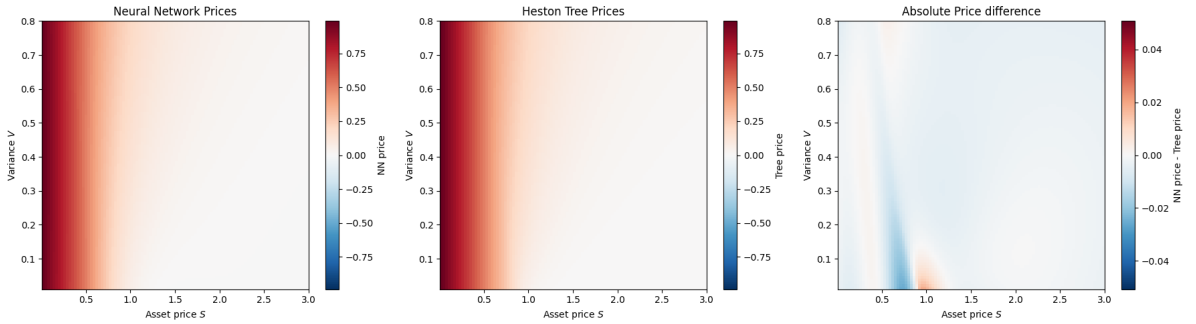

Figure 4.5 American put option prices by the PINN and the tree across S and V parameters at time $t = 0$

We can see that the neural network indeed reproduces the solution approximated by the binomial tree faithfully, even though it tends to perform less well at the money especially at lower volatilities. We can take slices of our three dimensional heatmap on the triple (t, S, V) to get a better understanding of the performance, by fixing two of the variables at the default $(t = 0, S = 1.0, V = 0.1)$ and plotting the option price against the other variable. This is shown in Figure 4.6.

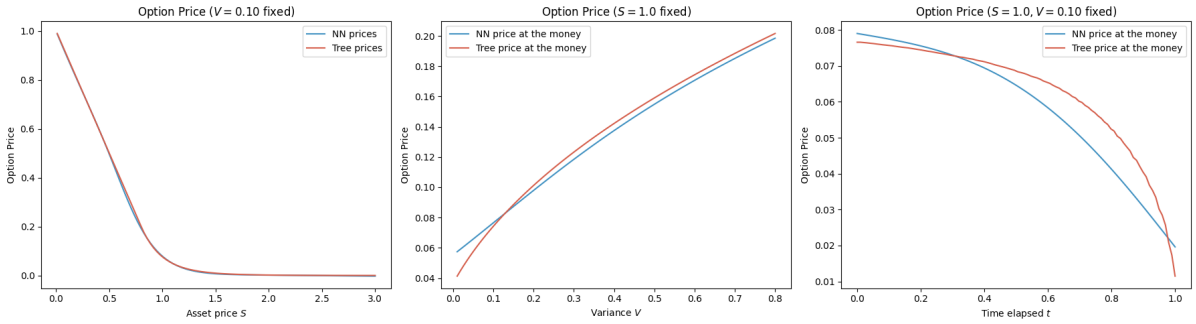

Figure 4.6 PINN vs tree prices varying across S, V and t parameters

We can see that the PINN does indeed capture well the general shape and behaviour of the option price, especially for varying across S and V but it struggles a bit more with the term-structure (varying t). That said, it is still able to capture the theta decay with its monotonicity, which is a good sign. Overall, the results are very promising for the one-dimensional case, with the PINN capable of capturing the option price across the whole input domain with good accuracy.

4.5.3 Multi-asset PINN solution

The multi-asset case is more difficult to train, as we have a higher dimensional input space and require a greater batch size. We choose 10000 to be the batch size, with the same learning rate and step size 1500 and gamma 0.9 for the scheduler, with an L-BFGS fine-tuning step. The training takes several hours to complete and hence manual tuning and iteration of the model are slow, and we will only show the results of one instance.

As discussed before, we will set our parameters satisfying the conditions required for a one-dimensional reduction (cross-asset correlations equal to zero). The left figure of Figure 4.7 shows the heatmap of the option prices across the S^1-S^2 plane with the time fixed at $t = 0$ and initial variance fixed at $V_0 = 0.04$.

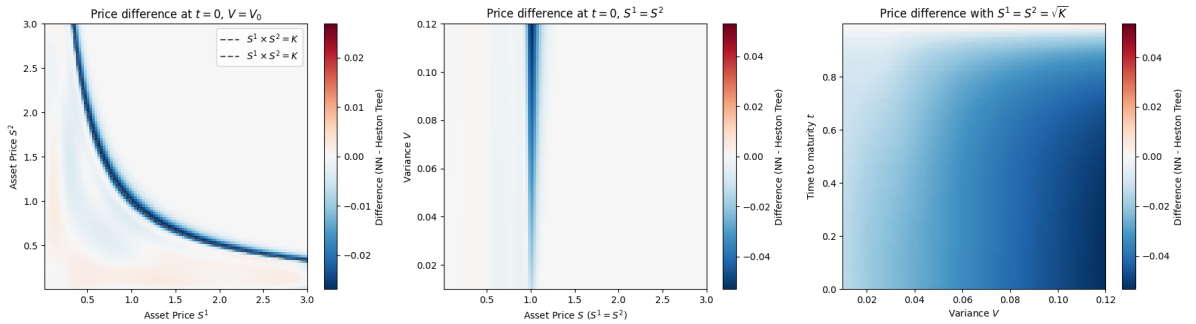

Figure 4.7 Heatmap of the difference between PINN and tree prices

We can see that the fit is good in-the-money and out-of-the-money but struggles much more at the money (when $S^1 \times S^2 = K$, indicated with the black dotted line), which is expected as the option price is more sensitive, and we are now in a four-dimensional input space and hence accuracy is more difficult to achieve. We will zoom in on the at-the-money region by fixing $t = 0$ and $S^1 = S^2 = \sqrt{K}$ and plotting the option price across V, which is shown in the middle figure of Figure 4.7.

We can see that the weakness of the PINN is along the strike price with $S^1 = S^2 = \sqrt{K}$, and

the accuracy is better for lower volatilities. Overall, the results are okay, given that we are sacrificing some accuracy in order to be able to price along the whole input domain, but we will expect the PINN to struggle more with estimating free boundaries correctly since there is a bias in the prices already.

Finally, we would like to check how the PINN captures the price relationship between different initial variances and time to maturity, which is shown in the right figure of Figure 4.7 where we plot the option price across t and V with $S^1 = S^2 = \sqrt{K}$. We can see that this relationship is captured less well: the PINN prefers to give roughly the same price across this cross-section, which is a side-effect of our loss function construction. Since the PDE loss is averaged across the domain, the network settles for giving the average price across this cross-section as it is easier to satisfy the PDE in this way, and the boundary conditions are not strong enough to force the network to learn the correct relationship across this cross-section. To remedy this, we should give more attention to how the weights of the loss components are set, which requires a lot of tuning and experimentation, which is less feasible in this case with each model taking several hours to train, but is something that could be further explored with more computational resources.

Even though the accuracy of the option price for the multi-asset case is not perfect, there is still some value in estimating the free boundary for the multi-asset case, which we will do in the next section.

4.5.4 Free boundary

One-dimensional case

We will first present the free boundary for the one-dimensional case. As this is a boundary on three dimensions, we will present it as a cross section, where one variable will be fixed at its default value ($t = 0, S = 1.0, V = 0.04$) and the other two will be plotted against each other. This would be cleaner and more interpretable than a colour-coded 3D plot. We will omit the t-S plane as it will look very similar to the free boundary for the Black-Scholes case 3.15, and we will also omit the t-V plane as it is not very informative, with continuation probabilities close to 1 for the whole region near the money at initial time regardless of the initial variance. This leaves us the free boundary on the S-V plane, shown in Figure 4.8.

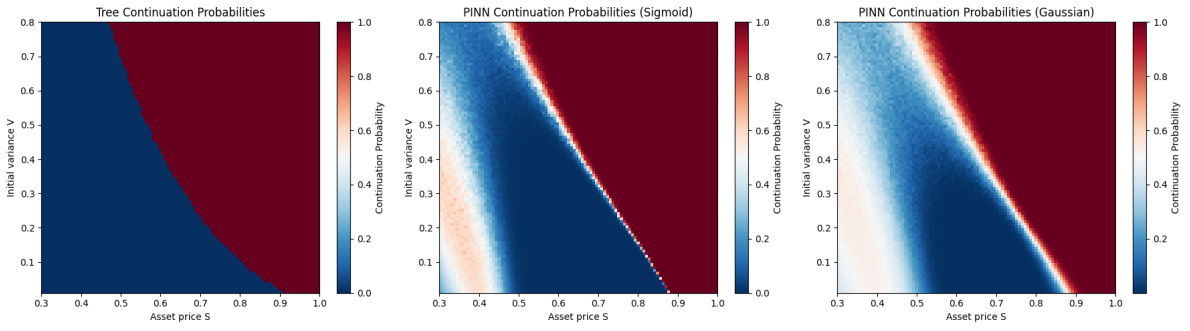

Figure 4.8 Continuation probabilities for the American put option on the S-V plane

We can see that the continuation region is reasonably well approximated with both methods, and we additionally observe some noise near the boundary which is due to the Monte Carlo estimation of the continuation value. Unfortunately, the network struggles to capture the curvature characteristic of the free boundary, which is unsurprising as it doesn't price the option with perfect accuracy in all areas. That said, the results are promising against the benchmark,

even though we have a patch of the continuation probabilities that are closer to one further in-the-money (at around $(S, V) = (0.4, 0.3)$), which is due to the bias in the option price in the corresponding region when we had our price comparison heatmaps in Figure 4.5.

Multi-asset case

We will first present the free boundary on the S^1-S^2 plane, where we fix $V = V_0$ and $t = 0$, as shown in Figure 4.9. This should be somewhat analogous to the Black-Scholes multi-asset free boundary as observed in Figure 3.17.

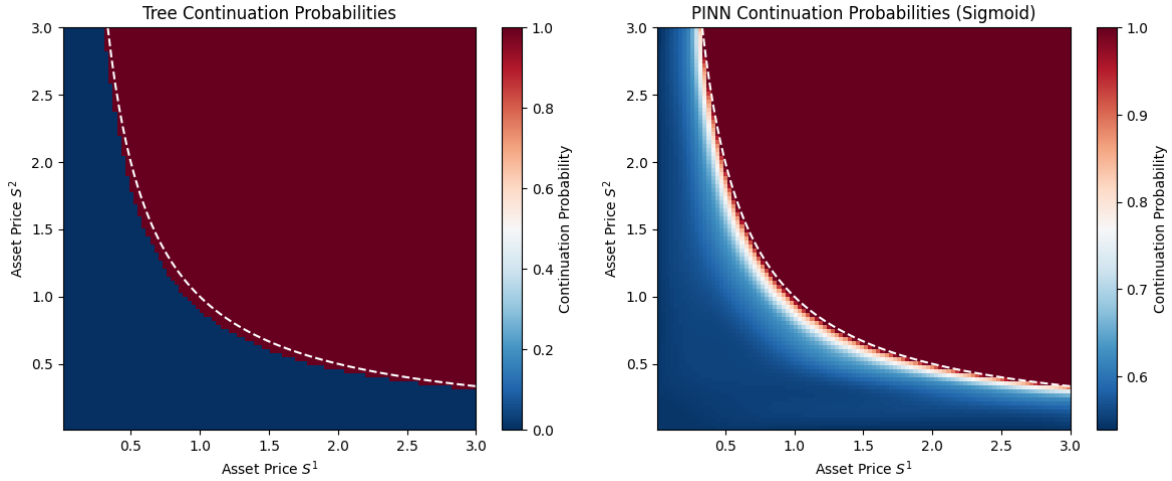

Figure 4.9 Continuation probabilities for the American put option on the S^1-S^2 plane

We can see that the continuation region is well approximated, with the shape correctly captured and the continuation probabilities resembling that of the tree method. This is promising: the PINN is able to capture the free boundary well with respect to the asset price.

We now present the free boundary on the S-t plane in Figure 4.10, where we fix $S^1 = S^2 = \sqrt{K}$ and $V = V_0$. This should be somewhat analogous to the free boundary for the one-dimensional case as observed in Figure 3.18. We first notice that the Heston tree free boundary is no longer smooth: this is likely due to the fact we are using an interpolation method which has some degree of error. The PINN also noticeably struggles a bit more to capture the curvature of the free boundary.

These shortcomings are likely linked to the limitations of the model, which has settled for an average price prediction across the time cross section, as an efficient and easy way of minimising the loss. To remedy this, we would need to do further experimentation with different training conditions. Even after changing the activation function between Sigmoid, Tanh and ReLU, trying different weights for the loss components, and even adding dropout layers, the results seem to be somewhat similar, so further improvements to the free boundary estimation would require a more fundamental change to the model, whether it be increasing its capacity or the sampling algorithm.

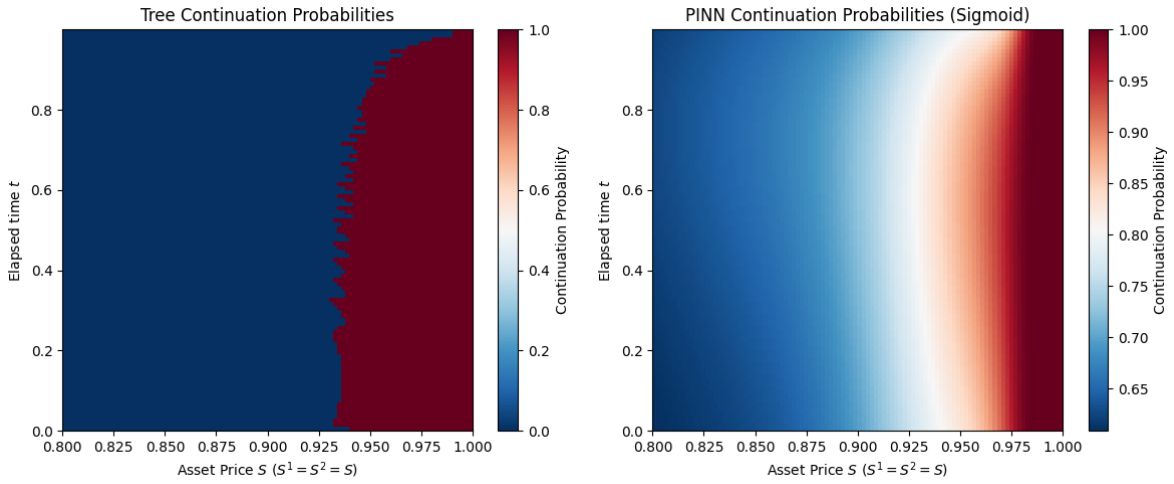

Figure 4.10 Continuation probabilities for the American put option on the S-t plane

Chapter 5

Conclusion

In this project, we have priced American put options on both the Black-Scholes and Heston models with various techniques, building benchmarks in the process to evaluate the validity of our results. The project mainly focuses on the use of variational methods to solve the variational inequality associated with the American option pricing problem, with analysis on the numerical results for their strengths and shortcomings. This project builds upon pre-existing techniques such as the tree and the PINN method, and adapts them to suit the American option pricing problem, while also providing a comprehensive mathematical justification for these numerical methods. We go beyond the existing literature by applying the variational methods to the problem of finding the free boundary where we develop a novel soft classification scheme, as well as constructing a solid foundation in our benchmarking process as a sanity check for our results.

We can see that the variational methods are very versatile: they are simply a matter of changing the underlying PDE and modifying the existing boundary conditions to apply the same method to a different model. Compare this with the tree method we used for the benchmark: the addition of a stochastic process for the variance in the Heston model requires a disproportionate amount of extra effort and care to construct a recombining model. The variational methods are also more efficient than the tree method: once trained, the neural network can be evaluated at any point in the domain very efficiently as it is only a forward pass, while the tree method requires the construction of the entire tree.

The versatility of the variational methods is underpinned by the exploration of the behaviour of the free boundary of the put on two assets after changing the payoffs. Even though we do not have a valid benchmark (a tree method would be difficult to construct, and a closed-form solution is of course not available), we can see that the free boundary behaves as expected. Granted, our model sacrifices some accuracy as we scale up the complexity and the number of dimensions, but the results are still reasonable, also taking into consideration hardware constraints.

A drawback of the variational methods is that they are not as accurate as the tree method, since they rely on approximating the solution with only the governing PDE and boundary conditions to guide the training. We can observe systematic bias in the results in different parts of the domain space, leading to less desirable results in the free boundary approximation with the Gaussian assumption, where the unbiasedness assumption is violated. Additionally, the variational methods are not as interpretable as the tree method, which has a clear structure in its discretisation of the underlying stochastic process and its pricing architecture is more transparent with its backwards induction. As a result, the variational methods are much more difficult to debug and can actually be relatively unstable to train, as tuning various hyperparameters and changing the activation function can lead to very different results. This brings into ques-

tion the feasibility of this method in the practical world, where the lack of interpretability and stability can be a problem for risk management, as we do not, of course, have any benchmarks to evaluate the results against.

For future work, it would be interesting to explore yet more complex models: we have shown that the variational method is versatile enough to be applied to any model with any payoff, so we could introduce either a different volatility process or add a stochastic interest rate. Additionally, the work on the multi-dimensional pricing problem could be further refined, where we take advantage of a GPU to train a larger and more capable neural network across a longer training time, which would hopefully lead to better results. As lattice methods suffer from the curse of dimensionality, we could consider using the Least-Squares Monte Carlo method as a benchmark, which is more efficient in higher dimensions. Finally, the work on the free boundary approximation could be further refined, where we work on better soft classification models and more complex Monte Carlo schemes to retrieve the continuation value, which would hopefully allow us to observe some interesting shapes in complex payoffs in higher dimensions.

Bibliography

[1] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of political economy*, 81(3):637–654, 1973.

[2] S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The review of financial studies*, 6(2):327–343, 1993.

[3] J. C. Cox, S. A. Ross, and M. Rubinstein. Option pricing: A simplified approach. *Journal of financial Economics*, 7(3):229–263, 1979.

[4] M. Vellekoop and H. Nieuwenhuis. A tree-based method to price american options in the heston model. *The Journal of Computational Finance*, 13(1):1–21, 2009.

[5] P. Glasserman. *Monte Carlo methods in financial engineering*. Springer, 2004.

[6] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[7] Y. Bai, T. Chaolu, and S. Bilige. The application of improved physics-informed neural network (ipinn) method in finance. *Nonlinear Dynamics*, 107(4):3655–3667, 2022.

[8] Y. Zhao and H. Zheng. Fractional-boundary-regularized deep galerkin method for variational inequalities in mixed optimal stopping and control. *arXiv preprint arXiv:2505.19309*, 2025.

[9] I. Karatzas and S. Shreve. *Brownian motion and stochastic calculus*. springer, 2014.

[10] I. Karatzas, S. E. Shreve, I. Karatzas, and S. E. Shreve. *Methods of mathematical finance*, volume 39. Springer, 1998.

[11] N. Wiener. Differential-space. *Journal of Mathematics and Physics*, 2(1-4):131–174, 1923.

[12] H. Pham. *Continuous-time stochastic control and optimization with financial applications*, volume 61. Springer Science & Business Media, 2009.

[13] Ömer Deniz Akyıldız. Stochastic simulation (lecture notes), 2025.

[14] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.

[15] Z. Hu, K. Shukla, G. E. Karniadakis, and K. Kawaguchi. Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks*, 176:106369, 2024.

[16] A. Dhiman and Y. Hu. Physics informed neural network for option pricing. *arXiv preprint arXiv:2312.06711*, 2023.

[17] S. Wang, S. Sankaran, H. Wang, and P. Perdikaris. An expert's guide to training physics-informed neural networks. *arXiv preprint arXiv:2308.08468*, 2023.

[18] S. J. Reddi, S. Kale, and S. Kumar. On the convergence of adam and beyond. *arXiv preprint arXiv:1904.09237*, 2019.

[19] J. Nocedal and S. J. Wright. *Numerical optimization*. Springer, 2006.

[20] S. Markidis. The old and the new: Can physics-informed deep-learning replace traditional linear solvers? *Frontiers in big Data*, 4:669097, 2021.

[21] D. C. Liu and J. Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.

[22] S. E. Shreve et al. *Stochastic calculus for finance II: Continuous-time models*, volume 11. Springer, 2004.

[23] H. Zheng. Fundamentals of options pricing (lecture notes), 2025.

[24] S.-S. Jo et al. Variational inequality for perpetual american option price and convergence to the solution of the difference equation. *arXiv preprint arXiv:1903.05189*, 2019.

[25] R. A. Adams and J. J. Fournier. *Sobolev spaces*, volume 140. Elsevier, 2003.

[26] E. Di Nezza, G. Palatucci, and E. Valdinoci. Hitchhiker's guide to the fractional sobolev spaces. *Bulletin des sciences mathématiques*, 136(5):521–573, 2012.

[27] Y. Zang, G. Bao, X. Ye, and H. Zhou. Weak adversarial networks for high-dimensional partial differential equations. *Journal of Computational Physics*, 411:109409, 2020.

[28] Y. Zhao and H. Zheng. Neural network convergence for variational inequalities. *arXiv preprint arXiv:2509.26535*, 2025.

[29] J. L. Lions and E. Magenes. *Non-homogeneous boundary value problems and applications: Vol. 1*, volume 1. Springer Science & Business Media, 2012.

[30] J. Detemple. *American-style derivatives: Valuation and computation*. Chapman and Hall/CRC, 2005.

[31] J. Gatheral. *The volatility surface: a practitioner's guide*. John Wiley & Sons, 2011.

[32] D. Duffie, J. Pan, and K. Singleton. Transform analysis and asset pricing for affine jump-diffusions. *Econometrica*, 68(6):1343–1376, 2000.

[33] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert. The little heston trap. *Wilmott*, (1): 83–92, 2007.

[34] J. Gil-Pelaez. Note on the inversion theorem. *Biometrika*, 38(3-4):481–482, 1951.

[35] W. H. Press. *Numerical recipes 3rd edition: The art of scientific computing*. Cambridge university press, 2007.

[36] P. Bressloff. Introduction to stochastic differential equations and stochastic processes (lecture notes), 2025.